

# Spring 面试题速记通关版 \_ 面试刷题 mianshiya.com

本资源来自面试鸭: <https://www.mianshiya.com>

推荐更多免费学编程资源:

1. 编程导航学习网站: [学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库: [开始刷题, 面试遇原题!](#)
3. 精选简历模板大全: [1 分钟搞定简历!](#)
4. AI 资源导航网站: [获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试: [随时随地提升面试能力](#)

## 在 Spring 中, 拦截器和过滤器有什么区别?

拦截器和过滤器都能在请求处理前后做点事, 但它们的出身、管辖范围和能力不太一样。

过滤器是 Servlet 规范的一部分, 由容器管理, 只要符合 Servlet 规则的 Web 应用都能用。它在请求到达 Spring 容器之前就起作用了, 比如处理字符编码、日志记录、跨域等脏活累活。它的执行不依赖 Spring, 所以你压根不经过 Spring 的时候它也能工作。

而拦截器是 Spring MVC 自己搞的一套机制, 归 Spring 管。它只能拦截进入 Controller 的请求, HandlerInterceptor 接口提供了 preHandle、postHandle 和 afterCompletion 三个钩子。因为它在 Spring 容器里, 所以能直接拿到 Bean, 比如注入一个 Service 做权限校验。

1) 过滤器先执行, 覆盖所有 web 请求, 包括静态资源 (除非你配置排除) 2) 拦截器后执行, 只对 Spring MVC 映射的路径生效 3) 过滤器通过 FilterChain 控制流程, 要记得调用 chain.doFilter() 4) 拦截器返回布尔值决定是否继续, 更直观

代码上, 过滤器需要实现 Filter 接口并注册到容器, 可以用 @WebFilter 或配置类; 拦截器实现 HandlerInterceptor, 再通过 WebMvcConfigurer 添加。



一般像安全认证这种需要全局控制的用过滤器, 业务相关的比如接口耗时统计、用户上下文绑定就用拦截器。

## Spring 中的 @Primary 注解的作用是什么?

当 Spring 容器里某个接口有多个实现类, 被 @Component 或者 @Service 注册成 Bean 时, 用 @Autowired 默认按类型注入就会搞不定, 因为容器发现一堆同类型的 Bean, 不知道该选哪个。

这时候 @Primary 就派上用场了。它用来标记 **首选 Bean**。只要某个实现类加上了这个注解, Spring 在自动装配时就会优先选它, 避免抛出 NoUniqueBeanDefinitionException。

比如你有两个数据源配置类都实现了 DataSource, 其中一个标了 @Primary, 那其他地方直接 @Autowired 就能注入这个主数据源, 不用额外指定 @Qualifier。

```
@Service
public class MySQLDataService implements DataService { }

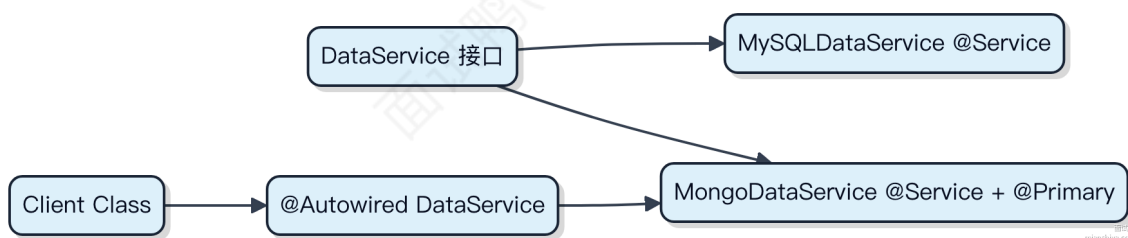
@Service
```

```
@Primary
public class MongoDataService implements DataService { }
```

上面这段代码里，虽然两个都是 `DataService` 的实现，但 `MongoDataService` 是首选项。其他类直接 `@Autowired DataService`，拿到的就是Mongo那个。

要注意的是，`@Primary` 和 `@Qualifier` 可以解决同样的问题，但思路不同。`@Qualifier` 是让调用方主动指名要哪个，适合需要**明确选择**的场景，比如 `@Qualifier("mysqlService")`；而 `@Primary` 是从定义端声明“我才是默认的”，更适合设定一个通用默认实现。

- 1) 一个接口多个实现时，Spring无法决定注入哪一个
- 2) `@Primary` 告诉容器：选我作为默认选项
- 3) 与 `@Qualifier` 配合使用更灵活，但两者目的不同



## Spring中的@Value注解的作用是什么？

`@Value` 是 Spring 里用来做属性注入的注解，特别适合把配置文件里的值塞到字段上。

它支持直接赋值，比如字符串、数字，也支持 SpEL 表达式。像配置中心的参数，`application.yml` 里的内容，都能通过它拿过来用。你写 `@Value("${server.port}")`，Spring 启动时就会去环境变量或配置文件里找这个 key，找到就注入进去。

如果找不到，默认会报错，但你可以给默认值，写成 `@Value("${server.port:8080}")` 就行，这样没配的时候自动用 8080。

- 1) 基本类型和字符串可以直接注入
- 2) 支持 `#{}`  写 SpEL，比如 `@Value("#{systemProperties['os.name']}")` 拿系统属性
- 3) 数组或集合可以通过逗号分隔的字符串注入，配合 `@Value("${list:value1,value2}")` 然后用 `String[]` 接收

注意它是在 Bean 初始化阶段由 `BeanPostProcessor` 处理的，具体是 `AutowiredAnnotationBeanPostProcessor` 负责扫描和注入逻辑。

不过别在静态字段上用 `@Value`，Spring 管不着静态成员，值压根不会被设置进去，写了也白搭。

现在主流都用 `@ConfigurationProperties` 做类型安全的配置绑定，`@Value` 更适合零散、简单的场景，比如某个开关标志。

## @Component、@Controller、@Repository和@Service的区别？

Spring 用注解做组件扫描时，这几个注解本质都一样，都是 `@Component` 的派生注解，容器都会把它们当 Spring Bean 来管理。真正起作用的是 **语义化** 和 **AOP 切面的针对性处理**。

- 1) `@Component` 是通用组件，适合那些不太能归类的工具类或配置类。

- 2) @Controller 用在 Web 层，比如 Spring MVC 的控制器。它会被组件扫描自动注册，并且通常配合 @RequestMapping 使用。框架会特别识别它，用于请求映射和异常处理。
- 3) @Repository 专用于数据访问层，比如 DAO 类。它的关键作用是让 Spring 能把数据库相关的异常（比如 SQLException）统一转换成 Spring 的 DataAccessException，这个事是由 **PersistenceExceptionTranslationPostProcessor** 做的，它默认只处理加了 @Repository 的类。
- 4) @Service 用在业务逻辑层，虽然框架本身对它没有特殊处理，但团队约定俗成用来标识服务类，方便代码结构清晰，也便于 AOP 针对 service 包做事增强。

```
@Repository
public class UserDAO {
    public void save(User user) { /* ... */ }
}

@Service
public class UserService {
    @Autowired private UserDAO dao;
    // 业务方法上加 @Transactional 更自然
}
```

说白了，除了 @Repository 有异常翻译这层实际功能外，其他三个主要是为了分层清晰、便于维护和切面匹配。不错就行，别混着用。

### @Bean和@Component有什么区别？

@Bean 是方法级别的注解，通常写在配置类里，你得手动 new 对象出来交给 Spring 管。比如数据源这种需要复杂初始化逻辑的，用它最合适。

@Component 是类级别的，加在类上，Spring 自动扫描到就会把这个类当 bean 创建，适合那些不需要特别定制构造过程的普通组件。像 Service、Controller 这些层，直接贴个注解就完事了，省心。

- 1) @Bean 的控制权在方法内部，你可以精确控制对象怎么创建，比如设置参数、调用静态工厂。
- 2) @Component 靠 classpath 扫描生效，配合 @ComponentScan 一起用，自动发现组件。
- 3) 它们最终都是把实例注册进 IOC 容器，但 **生命周期管理方式** 不一样。

举个例子：你想集成 RedisTemplate，可能要先配 JedisConnectionFactory，再 set 到 template 上。这种链式配置，写在 @Bean 方法里一目了然。

```
@Configuration
public class RedisConfig {
    @Bean
    public RedisTemplate<String, Object> redisTemplate() {
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(connectionFactory());
        return template;
    }
}
```

而一个订单服务 OrderService，本身没啥初始化逻辑，直接用 @Component：

```
@Component
public class OrderService { ... }
```

其实选哪个，关键看你要不要干预对象的构建过程。需要精细控制就用 @Bean，否则 @Component 更轻量。

## @Qualifier 注解有什么作用

当 Spring 容器里同一个接口有多个实现类时，光用 @Autowired 会懵，不知道该注入哪一个。这时候就得靠 @Qualifier 来指名道姓。

比如你有个接口叫 UserService，底下有 LocalUserService 和 RemoteUserService 两个实现，都被扫进容器了。你在 Controller 里要注入远程那个，就得这么写：

```
@Autowired
@Qualifier("remoteUserService")
private UserService userService;
```

这里的 "remoteUserService" 是 Bean 的名字，一般就是首字母小写的类名。Spring 凭这个名字精准定位到该注入哪个实例。

反过来，如果不用 @Qualifier，又确实存在多个候选 Bean，应用启动直接报错，抛 NoUniqueBeanDefinitionException。所以这个注解是解决同类型多实例依赖注入歧义的标配方案。

实际项目里像数据源配置、不同策略实现（比如支付方式）、多套客户端实例都会碰到这情况，@Qualifier 就是来兜底的。

- 1) 它和 @Primary 能搭配用，@Primary 标记默认选哪个，@Qualifier 用于特殊指定
- 2) 也可以结合自定义注解，把 @Qualifier 封装掉，代码更清晰，比如定义个 @RemoteService 注解
- 3) 别忘了，bean name 写错了会找不到，IDE 一般能帮你校验，但手动写字符串还是容易翻车

## Spring 中的 @ModelAttribute 注解的作用是什么？

@ModelAttribute 主要用来预处理请求参数，把数据提前塞进 Model 里，让后续方法能直接用。

- 1) 它可以加在方法上，这个方法会在每个 @RequestMapping 方法执行前先跑一遍。比如你有个表单要用到省份列表，就可以用它从数据库捞出来放进 Model，这样页面自动能访问到。

```
@ModelAttribute
public void populateStates(Model model) {
    model.addAttribute("states", stateService.getStates());
}
```

- 2) 它也能加在控制器方法的参数上，表示这个参数不是从 URL 或 body 来的，而是从 Model 里找对应名字的对象，或者通过参数解析器自动组装。常用于表单提交时绑定对象。

```
@PostMapping("/users")
public String saveUser(@ModelAttribute User user) {
    // user 已经由请求参数自动填充
    userService.save(user);
}
```

```
        return "redirect:/users";  
    }
```

注意一点，如果 Model 里没有现成的 User 实例，Spring 会调用它的无参构造函数创建一个，然后再用请求里的 name 、 email 这些字段去 set。这个过程**压根不经过 @RequestBody 那套反序列化流程**，是靠数据绑定器 WebDataBinder 搞定的。

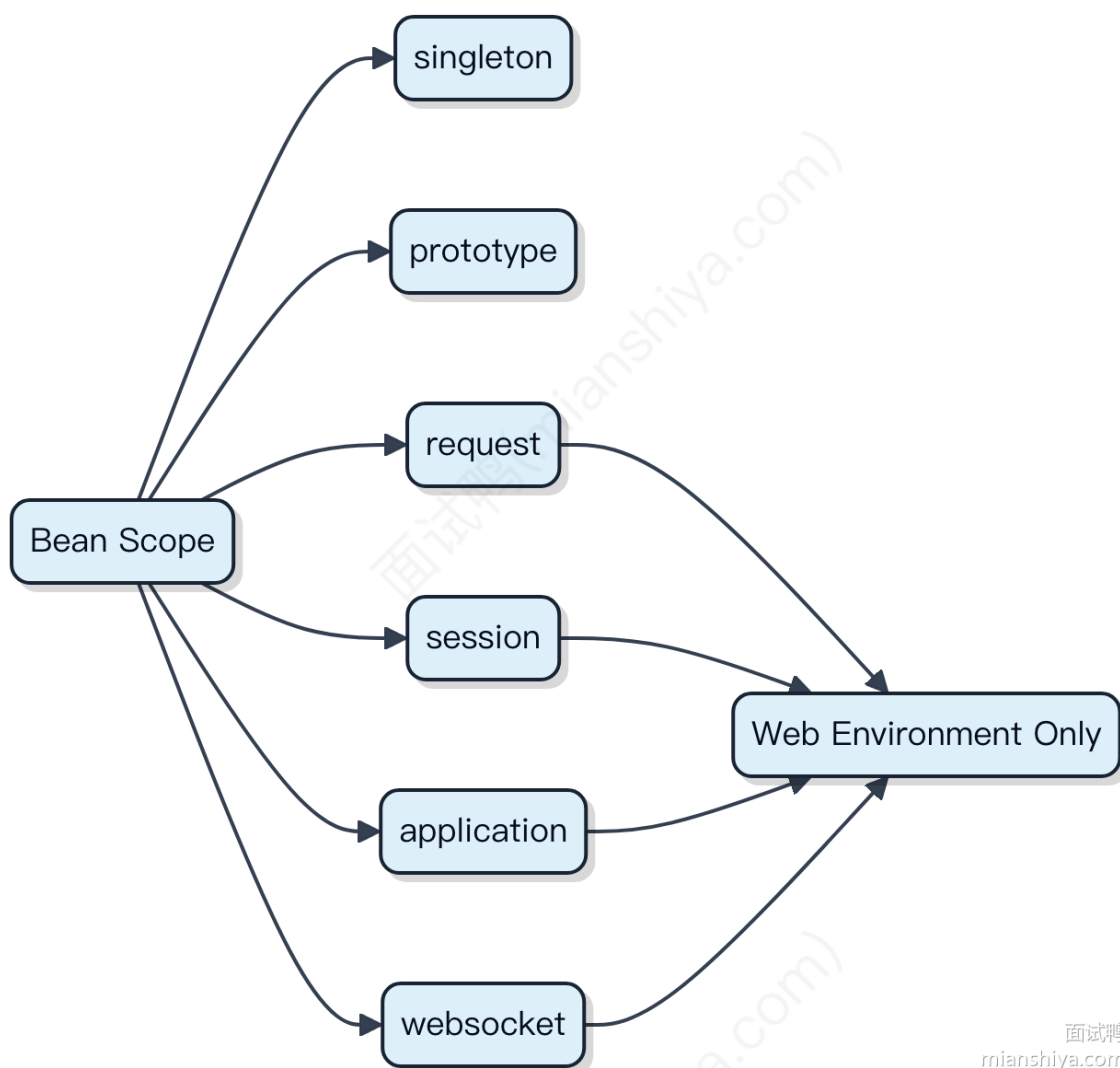
一般搭配 @SessionAttributes 用，可以把表单对象跨请求存住，比如多步骤表单。但别滥用，容易把 Model 塞得太满，调试起来头疼。

## Spring Bean 一共有几种作用域？

Spring 里的 Bean 作用域，说白了就是这个实例归谁管、活多久。容器默认给你管到底的叫单例，但实际场景远不止这一种。

- 1) singleton 是最常用的，整个应用就一个实例，Spring 容器启动时创建，销毁容器时才释放。像配置类、工具类这种无状态组件用它最合适。
- 2) prototype 每次拿都给你 new 一个新对象，不缓存。适合有状态的 Bean，比如某个请求级别的数据处理器，每次都需要干净的上下文。
- 3) request 和 session 是 Web 环境专属。request 级别的 Bean 在一次 HTTP 请求中共享，请求结束就销毁；session 级别的则绑定到用户会话，用户退出或 session 过期才清理。在 Spring MVC 或 Spring Boot 的 Web 项目里才能用。
- 4) application 作用于 ServletContext 生命周期，整个 Web 应用共享一份实例，基本不常用。
- 5) websocket 是 Spring 3.1 后加的，每个 WebSocket 会话拥有独立的 Bean 实例，用于 WebSocket 的会话隔离场景。

非 Web 项目里 request、session 这些直接用不了，跑起来会报错。所以选作用域得看运行环境和对象生命周期需求，别一股脑全用 singleton。



面试鸭  
mianshiya.com

### Spring 中的 @Validated 和 @Valid 注解有什么区别？

Spring 里的校验注解看着差不多，其实分工明确。@Valid 是 JSR-303 规范原生的，支持嵌套对象校验，但用在方法参数上时，Spring 不会自动触发它对方法入参的校验。

@Validated 是 Spring 自己加的，专为解决这个痛点。它能在类上做开启校验的开关，比如 Controller 上加了 @Validated，里面的方法参数就能配合 @Valid 起效。更重要的是，它支持分组校验，比如新增和修改用不同的校验规则。

- 1) @Valid 可以用在成员变量、方法参数、构造器参数上，适合嵌套对象校验
- 2) @Validated 只能用在方法和类型上，不能标在字段上
- 3) @Validated 支持分组，比如 @Validated(Update.class) 只跑更新组的校验
- 4) Spring MVC 方法参数前加 @Valid，得靠 AOP 拦截才能生效，而 @Validated 是触发点

举个例子，Controller 方法参数是对象，里面还有个 List

，Address 里也有 @NotBlank。这时候主对象用 @Valid 才会递归校验到每一项，只用 @Validated 不会深入集合内部。

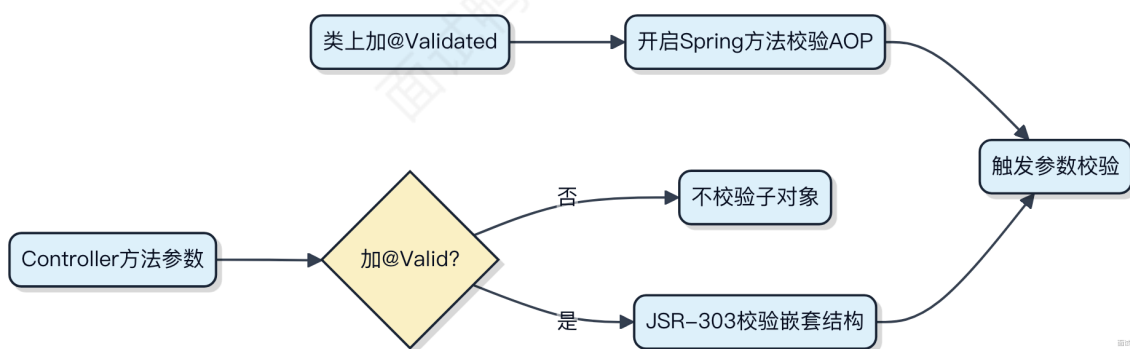
代码上常见写法：

```
@PostMapping("/user")
public String save(@Valid @RequestBody User user) { ... }
```

如果要用分组：

```
public interface Update {}
public interface Create extends Default {}

@PutMapping("/user")
public String update(@Validated(Update.class) @RequestBody User user) { ... }
```



### Spring 事务在什么情况下会失效？

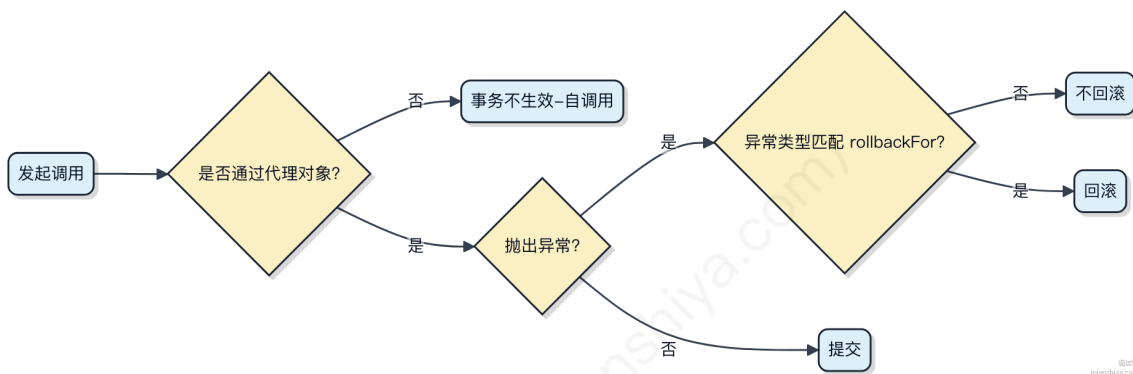
Spring 事务失效最常见的原因是调用发生在同一个类内部，导致 AOP 代理没生效。比如你在一个 `@Service` 里写了个 `methodA()` 调用了同样加了 `@Transactional` 的 `methodB()`，这时候事务压根不经过代理对象，**自调用**直接绕过了切面。

另一个典型是异常被吞了或者捕获后没抛出去。Spring 默认只对 `RuntimeException` 和 `Error` 回滚，如果你 `catch` 了 `Exception` 又不主动声明 `rollback`，事务就不会回滚。想让它对普通异常也回滚，得加上 `@Transactional(rollbackFor = Exception.class)`。

还有就是方法不是 `public` 的。AOP 基于代理，非 `public` 方法不会被 CGLIB 或 JDK 动态代理拦截，事务自然失效。别试图在 `private` 方法上加事务注解，没用。

数据库引擎不支持事务也会翻车。比如 MySQL 用 MyISAM 引擎，压根不支持事务，你代码写得再漂亮也没用。生产环境基本都用 InnoDB，但有时候配置疏忽可能踩坑。

传播行为设置不当也可能“看起来像失效”。比如当前方法已经处于事务中，而你用了 `PROPAGATION_SUPPORTS`，它会跟着走，但外部没事务时它也不创建，这时候逻辑就变了。



简单说，只要记住三点：走代理、抛对异常、数据库撑住。这三块不出问题，事务基本能稳住。

### @Async 如何避免内部调用失效?

Spring 的 @Async 注解失效，最常见的就是方法内部调用导致 AOP 代理没生效。你写了个异步方法，但同一个类里另一个方法直接调它，这时候压根不经过代理对象，异步就没了。

- 1) 核心是 **代理机制** 没触发。@Async 依赖 Spring AOP，只有外部 bean 调用才会走代理，内部方法直调等于普通方法调用。
- 2) 解决方案一：把异步方法拆到另一个 @Service 里，通过注入的方式调。这是最干净的做法，职责也清晰。

```
@Service
public class AsyncTaskService {
    @Async
    public void sendEmail() { /* 异步发邮件 */ }
}

@Service
public class UserService {
    @Autowired private AsyncTaskService taskService;

    public void register() {
        // 外部调用，走代理
        taskService.sendEmail();
    }
}
```

- 3) 方案二：自己注入自己，通过代理对象调。虽然有点绕，但在无法拆分时可用。

```
@Autowired private UserService self;

public void register() {
    self.sendEmail(); // 通过代理调自己
}
```

- 4) 方案三：用 ApplicationContext 拿代理对象，或者 AopContext.currentProxy()，但得配置 expose-proxy="true"，代码侵入性强，一般不推荐。



其实最稳妥的还是设计上避免自调用，把异步逻辑下沉到独立组件。像定时任务用 `@Scheduled`、事件驱动用 `Spring Event`，都比硬怼 `@Async` 更可控。

## Spring 中的 @Profile 注解的作用是什么？

根据运行环境激活不同配置，让应用能适应开发、测试、生产等不同场景。比如数据源在本地连 H2，在线上连 MySQL，`@Profile` 就是做这个开关的。

它标记在 `@Component` 或 `@Configuration` 类上，Spring 容器启动时会根据当前激活的 profile 决定是否加载这些组件。没被匹配类直接跳过，压根不经过初始化流程。

- 1) 通过 `@Profile("dev")` 指定该配置仅在 dev 环境生效
- 2) 多个环境用 `@Profile({"dev", "test"})`，满足其一即可
- 3) 取反逻辑用 `@Profile("!prod")`，非生产环境都加载

激活方式常见两种：JVM 参数 `-Dspring.profiles.active=prod`，或者配置文件里写 `spring.profiles.active=prod`。环境变量也行，优先级看具体部署方式。

```
@Configuration
@Profile("dev")
public class DevDataSourceConfig {
    @Bean
    public DataSource dataSource() {
        return new HikariDataSource(); // 本地用轻量数据库
    }
}
```

和 `@Conditional` 相比，`@Profile` 更聚焦环境维度判断，属于它的特例场景。真要搞复杂条件控制，还是得上 `@Conditional` 自定义规则。

## @Async 什么时候会失效？

方法调用在同一个类中会导致代理失效，这是最常见的情况。Spring 的 `@Async` 基于 AOP 代理实现，只有外部 bean 调用方法时才会走代理逻辑，进而触发异步执行。

如果本类方法直接调用被 `@Async` 标记的方法，相当于 `this.method()`，压根不经过代理对象，异步机制就失效了。

- 1) 确保调用方和被调用方是不同的 Spring Bean
- 2) `@Async` 方法不能是 `private`、`static` 或 `final` 的，代理无法覆盖这些方法
- 3) 所在类必须被 Spring 容器管理，普通 `new` 出来的对象无效
- 4) 需要启用 `@EnableAsync`，否则注解不生效
- 5) 异常未处理可能导致线程池任务中断，表面看像是“没执行”

代码示例：

```
@Service
public class AsyncTaskService {
    @Async
    public void sendEmail() {
        System.out.println("当前线程: " + Thread.currentThread().getName());
    }
}
```

```

}

@Service
public class BusinessService {
    @Autowired
    private AsyncTaskService taskService;

    public void process() {
        taskService.sendEmail(); // 正确: 通过代理调用
    }
}

```

错误用法就是把 `taskService` 的调用换成 `this.sendEmail()`。

## Spring MVC 中如何处理异常?

Spring MVC 的异常处理机制其实挺清晰的，关键在于分层拦截和统一出口。你不需要到处 `try-catch`，框架已经帮你设计好了几道防线。

第一道防线是 `@ExceptionHandler`，它写在 `Controller` 内部，能处理当前类中方法抛出的异常。如果想跨多个 `Controller` 复用，就把这个方法提到一个类里，并加上 `@ControllerAdvice`。这样一来，全局的异常比如参数校验失败、业务逻辑异常，都能集中捕获。

比如你调用接口时传了非法参数，Spring 会抛 `MethodArgumentNotValidException`，你可以在 `Advice` 类里专门捕获它，返回一个结构化的错误 JSON，前端接起来也方便。

- 1) `@ControllerAdvice` 可以配合 `@ExceptionHandler` 做全局异常处理
- 2) 支持按异常类型定义多个处理方法，精度可控
- 3) 最终返回值通常封装成统一响应体，比如 `{code: 500, msg: "服务器异常", data: null}`

还有一种情况是框架底层抛的异常，比如 `DispatcherServlet` 初始化失败，这种压根不经过 `Controller`，就得靠 `HandlerExceptionResolver` 接口来兜底。不过一般我们不用自己实现，Spring 已经内置了几个，像 `DefaultHandlerExceptionResolver` 就能把 Spring 内部异常转成对应的 HTTP 状态码。



别忘了还有 `@ResponseStatus`，直接标在自定义异常上，触发时自动设置 HTTP 状态码，省得手动设。

## Spring MVC 中的国际化支持是如何实现的?

Spring MVC 的国际化主要靠 `LocaleResolver` 和 `MessageSource` 两个组件协作完成。

用户的语言偏好由 `LocaleResolver` 解析，常见的有基于请求头的 `AcceptHeaderLocaleResolver`，也有基于 Cookie 或 Session 的实现。比如浏览器发来 `Accept-Language: zh-CN`，框架就知道该用中文。

真正加载语言包的是 `MessageSource`，通常用 `ResourceBundleMessageSource` 读取类路径下的 `messages_zh.properties`、`messages_en.properties` 这样的文件。你在 `controller` 或页面里通过 `code` 拿

文本，它会根据当前 locale 返回对应翻译。

代码里可以用 `@RequestHeader("Accept-Language")` 手动感知语言，但更常见的是直接注入 `MessageSource` 实例去 `getMessage`。

1) 配置示例：

```
@Bean
public MessageSource messageSource() {
    ResourceBundleMessageSource source = new ResourceBundleMessageSource();
    source.setBasename("messages");
    source.setDefaultEncoding("UTF-8");
    return source;
}
```

2) 目录结构：

```
src/main/resources/
  messages.properties      # 默认
  messages_zh.properties   # 中文
  messages_en.properties   # 英文
```

前端如果用 Thymeleaf，直接 `${#messages.text('login.title')}` 就能取到。REST 接口则通常在 service 层调用 `messageSource.getMessage(code, args, locale)` 组装响应。

整个流程不依赖任何特定中间件，**核心机制**就是“解析区域 + 动态查表”。只要语言文件放对位置，换语言基本不用改代码。

## Spring 中的 @PostConstruct 和 @PreDestroy 注解的作用是什么？

Spring 的这两个注解用来标记**生命周期回调方法**，让开发者能在 Bean 初始化完成或销毁前执行自定义逻辑。

1) `@PostConstruct` 标记的方法会在依赖注入完成后自动执行，只会触发一次。比如你有个数据缓存组件，需要在应用启动后立即加载基础数据，就可以放在这里：

```
@PostConstruct
public void init() {
    cache.loadAll();
}
```

这个时机点很关键：所有 `@Autowired` 字段都已经填充完毕，可以安全使用依赖项。

2) `@PreDestroy` 用在销毁前清理资源，仅适用于 singleton 和 prototype bean 在容器关闭时。典型场景是关闭线程池、释放文件句柄或通知注册中心下线：

```
@PreDestroy
public void cleanup() {
    executor.shutdown();
}
```

要注意的是，`@PreDestroy` 能正常触发的前提是应用能优雅关闭，比如通过 `ApplicationContext.close()` 或 JVM 的 shutdown hook。如果是 `kill -9` 这种粗暴方式，压根不经过 Spring 生命周期，脏活累活就搞不定了。

这两个注解本质上替代了实现 `InitializingBean` 和 `DisposableBean` 接口的方式，更轻量且不侵入业务代码。从执行顺序看，构造函数 → 依赖注入 → `@PostConstruct` → 业务方法 → `@PreDestroy`（容器关闭时）。

## Spring 中的 `@RequestBody` 和 `@ResponseBody` 注解的作用是什么？

这两个注解是 Spring MVC 处理 HTTP 请求和响应时最常用的工具，专门用来跟 JSON 打交道。

`@RequestBody` 的作用是把 HTTP 请求体里的 JSON 数据自动反序列化成 Java 对象。比如前端发来一个 JSON 字符串，Spring 会通过 Jackson 或 Fastjson 把它转成对应的 DTO 或实体类实例。没有它，你就得手动读取输入流再解析，麻烦不说还容易出错。

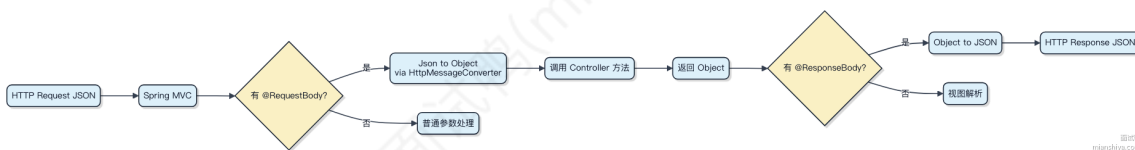
```
@PostMapping("/user")
public void createUser(@RequestBody User user) {
    // user 已经是解析好的对象了
}
```

反过来，`@ResponseBody` 是把方法返回的 Java 对象序列化成 JSON，写进 HTTP 响应体里。浏览器收到的就是标准的 JSON 数据。现在大多数接口都是前后端分离的 REST API，这个注解几乎是标配。

```
@GetMapping("/user/{id}")
@ResponseBody
public User getUser(@PathVariable Long id) {
    return userService.findById(id);
}
```

其实更常见的写法是直接用 `@RestController`，它本身就被 `@ResponseBody` 修饰了，所有方法默认都会返回 JSON。

这两个注解的背后依赖的是 **HttpMessageConverter**，Spring Boot 会自动配置好 Jackson 的实现。只要类路径下有 `jackson-databind`，这套机制就能跑起来，完全不用手写配置。



## Spring 中的 `@PathVariable` 注解的作用是什么？

`@PathVariable` 用来绑定 URL 模板中的变量到方法参数上。比如 RESTful 接口里常见的 `/users/123`，这个 123 就是路径变量。

Spring MVC 在处理请求时，会根据 `@RequestMapping` 或其衍生注解（如 `@GetMapping`）里的占位符，把对应位置的值提取出来，自动注入到加了 `@PathVariable` 的参数中。

1) 最简单的用法是直接匹配同名变量

```
@GetMapping("/orders/{id}")
public String getOrder(@PathVariable String id) {
    return "Order ID: " + id;
}
```

2) 如果参数名和路径变量名不一致，可以指定名称

```
@GetMapping("/categories/{cid}/products/{pid}")
public String getProduct(@PathVariable("cid") String categoryId,
    @PathVariable("pid") String productId) {
    return "Category " + categoryId + ", Product " + productId;
}
```

3) 支持类型转换，可以直接声明为基本类型或包装类

```
@GetMapping("/items/{itemId}")
public Item getItem(@PathVariable Long itemId) {
    return itemService.findById(itemId);
}
```

如果传的不是数字，会抛出 `TypeMismatchException`。

4) 变量可选？默认必须存在。如果想让某个路径变量可选，得配合 `required = false`

```
@GetMapping("/tags/{tagName}")
public String getTag(@PathVariable(required = false) String tagName) {
    if (tagName == null) return "All tags";
    return "Tag: " + tagName;
}
```

但注意，这种方式对 REST 设计不太友好，一般建议通过不同接口区分。

整个过程由 `HandlerMethodArgumentResolver` 的实现类 `PathVariableMethodArgumentResolver` 完成，属于 Spring MVC 参数解析机制的一环。

## Spring MVC 中的拦截器是什么？如何定义一个拦截器？

拦截器在 Spring MVC 里是用来对请求处理过程进行预处理和后处理的机制。它能在控制器方法执行前后、视图渲染前插入自定义逻辑，比如权限校验、日志记录、性能监控。

要定义一个拦截器，你需要实现 `HandlerInterceptor` 接口，重写它的三个方法：

- 1) `preHandle`：在控制器方法执行前调用，返回布尔值决定是否放行。返回 `false` 就会中断后续流程。
- 2) `postHandle`：控制器方法执行完但视图还没渲染时调用，可以用来修改模型数据或视图。
- 3) `afterCompletion`：整个请求完成（包括视图渲染）后调用，通常用于资源清理。

```
public class MyInterceptor implements HandlerInterceptor {
    @Override
```

```

    public boolean preHandle(HttpServletRequest request, HttpServletResponse
response, Object handler) {
        System.out.println("执行前");
        return true;
    }

    @Override
    public void postHandle(HttpServletRequest request, HttpServletResponse response,
Object handler, ModelAndView modelAndView) {
        System.out.println("执行后, 视图渲染前");
    }

    @Override
    public void afterCompletion(HttpServletRequest request, HttpServletResponse
response, Object handler, Exception ex) {
        System.out.println("请求完成");
    }
}

```

写好之后要在配置类中注册，继承 `WebMvcConfigurer`，把拦截器加到拦截路径里。比如你可以只拦截 `/api/**` 这样的接口路径。

注意这个 `handler` 参数是处理器方法的映射信息，不是 `Controller` 实例。如果要用注解判断权限，就在这里做反射解析。这种机制比过滤器更贴近 Spring 的调度流程，**粒度更细**，也更容易拿到 Spring 管理的 Bean。

### Spring 中的 `@ExceptionHandler` 注解的作用是什么？

`@ExceptionHandler` 是 Spring MVC 里专门处理控制器内部异常的注解。你把它加在方法上，一旦同一个 `Controller` 里其他方法抛出它能处理的异常类型，就会自动跳转到这个方法响应。

1) 最常见的用法是在一个 `Controller` 内部定义私有的异常处理逻辑。比如某个接口可能抛出 `IllegalArgumentException`，你就可以写：

```

@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<String> handleIllegalArgumentException() {
    return ResponseEntity.badRequest().body("参数不对");
}

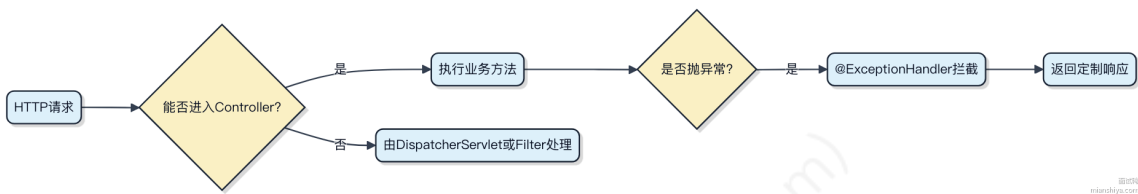
```

这样前端调用时传了非法参数，就不会直接返回 500，而是得到一个友好的 400 响应。

2) 但更实用的做法是结合 `@ControllerAdvice` 把异常处理做成全局的。把通用的异常捕获逻辑抽出去，所有 `Controller` 都能生效，避免每个类都重复写。

3) 它支持的返回类型很灵活，可以直接返回 `Model` 和 `View` 用于渲染错误页，也可以返回 `ResponseEntity` 构造 JSON 响应，适合前后端分离场景。

注意它只能捕获当前 `Controller` 或其调用链中抛出的异常。如果请求压根没进到 `Controller`，比如被拦截器拦住了，那它就搞不定了。这类全局性问题得靠 `@ControllerAdvice` + `@ExceptionHandler` 配合解决。



## Spring 中的 @ResponseStatus 注解的作用是什么?

@ResponseStatus 用在控制器的方法或异常类上，直接绑定 HTTP 状态码，省得你手动写 response.setStatus()。

比如抛出一个自定义异常，加上 @ResponseStatus(HttpStatus.NOT\_FOUND)，Spring 拦截到就会自动返回 404，连 controller 都不用碰。这招在全局异常处理里特别实用，配合 @ControllerAdvice 能统一响应状态。

- 1) 作用目标可以是方法，也可以是异常类
- 2) 状态码由 value 属性指定，比如 400、404、500
- 3) 可选 reason 字段，填充响应体里的错误信息

```
@ResponseStatus(value = HttpStatus.BAD_REQUEST, reason = "请求参数不对")
public class InvalidParamException extends RuntimeException {
}
```

注意它和 ResponseEntity 的区别：前者声明式，适合固定场景；后者编程式，灵活控制 body 和 header。要是两个都用了，ResponseEntity 优先级更高，毕竟运行时动态决定的嘛。

别滥用，在正常业务流里返回状态码，那该用 ResponseEntity 或 ResponseWriter 手动输出。@ResponseStatus 更适合“异常即状态”这种语义场景。

## Spring 中的 @RequestHeader 和 @CookieValue 注解的作用是什么?

用来从 HTTP 请求里提取特定的请求头和 Cookie 值，省得你手动通过 HttpServletRequest 去拿。

- 1) @RequestHeader 把请求头的值直接注入到方法参数里。比如要获取客户端支持的压缩类型：

```
@GetMapping("/info")
public String getInfo(@RequestHeader("Accept-Encoding") String encoding) {
    return "Encoding: " + encoding;
}
```

如果头部不存在，默认会抛异常，加上 required = false 就能避免：

```
@RequestHeader(value = "X-User-ID", required = false) String userId
```

- 2) @CookieValue 是专门取 Cookie 的。比如读取 JSESSIONID：

```
@GetMapping("/hello")
public String hello(@CookieValue("JSESSIONID") String jSessionId) {
```



```

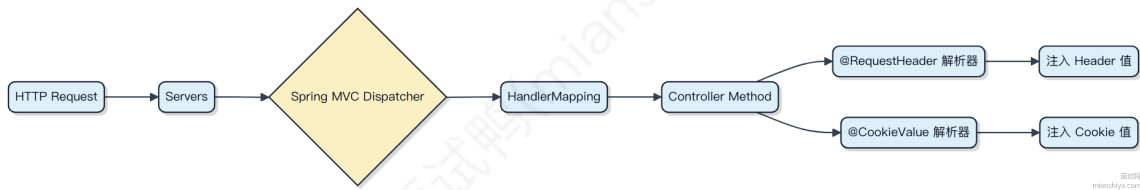
    return "Session ID: " + jSessionId;
}

```

也支持 `required = false` 和设置默认值 `defaultValue`。

这两个注解在写网关、鉴权逻辑时特别常见。像用 `@CookieValue` 拿 `token` 字段做免登录，或者通过 `@RequestHeader` 读取 `Authorization` 头传递 JWT（虽然更常见的做法是全局拦截器处理）。

注意它们只能用在控制器的方法参数上，属于 Spring MVC 的绑定机制的一部分。背后靠的是 **HandlerMethodArgumentResolver** 实现自动解析，整个过程对开发者透明。



### Spring 中的 `@SessionAttribute` 注解的作用是什么？

用来从当前 HTTP 会话中获取已存在的属性，通常用于处理用户登录状态或跨请求共享数据。它不是创建 session 属性的注解，而是假设 session 中已经有对应值，直接拿来用。

如果 session 中没有找到对应属性，默认会抛出异常，除非设置 `required = false`。这和 `@ModelAttribute` 类似，都是“取已有”的逻辑，而不是“存”或“创建”。

常见于需要强依赖登录用户信息的场景，比如管理员后台接口，要求用户必须已登录且 session 中有 `user` 对象。

```

@GetMapping("/profile")
public String getProfile(@SessionAttribute("user") User user, Model model) {
    model.addAttribute("user", user);
    return "profile";
}

```

- 1) 只能获取已经存在于 session 中的属性，不会自动创建
- 2) 适用于前后端不分离的传统 MVC 场景，比如 Thymeleaf 页面渲染
- 3) 在 Spring Security 集成时，常配合认证后的用户信息提取使用

现代 Web 应用越来越多采用无状态设计，像 JWT 这类方案让服务端不再维护 session，这种情况下 `@SessionAttribute` 用得越来越少。但在一些内部系统、企业应用里，基于 session 的认证还是主流，这时候它依然是个简单可靠的工具。

### Spring MVC 中如何处理表单提交？

表单提交在 Spring MVC 里其实就走标准的请求处理流程，关键在于怎么接数据、怎么转数据、怎么校验。

- 1) 前端发个 POST 请求到 `/submit`，Content-Type 一般是 `application/x-www-form-urlencoded`。Spring 的 **DispatcherServlet** 拿到请求后，根据 URL 找到对应的 `@Controller` 方法。
- 2) 方法参数通常用一个 `JavaBean` 接参，比如 `@ModelAttribute UserForm form`。Spring 会自动把表单字段按名字映射到对象属性上，支持基本类型、嵌套对象，甚至集合。如果字段名对不上，可以用



`@RequestParam("email")` 明确指定。

3) 想做数据校验? 加上 `@Valid` 就行。配合 `Hibernate Validator`, 用 `@NotBlank`、`@Email` 这些注解声明规则。校验失败的话, 会抛出 `MethodArgumentNotValidException`, 你可以用 `@ExceptionHandler` 统一处理, 返回错误信息给前端。

4) 如果表单带文件上传, 记得把 `Content-Type` 改成 `multipart/form-data`, 后端用 `MultipartFile` 接参。Spring 需要配置 `MultipartResolver`, 比如用 `CommonsMultipartResolver` 或基于 `Servlet 3.0` 的标准实现。

```
@PostMapping("/submit")
public String handleForm(@Valid @ModelAttribute UserForm form,
                          BindingResult result) {
    if (result.hasErrors()) {
        return "form-page"; // 返回页面, 带上错误
    }
    // 处理逻辑
    return "success";
}
```

整个过程压根不经过视图解析之前的拦截器也能介入, 比如打日志或者做权限检查。常见问题就是时间格式转换, 需要 `@DateTimeFormat(pattern = "yyyy-MM-dd")` 明确指定。

## Spring 中的 `@Scheduled` 注解的作用是什么?

定时任务的声明式配置, 用 `@Scheduled` 就能把一个普通方法变成周期性执行的任务, 不用手动去写线程调度逻辑。

1) 方法上加上 `@Scheduled`, 指定触发规则, Spring 容器启动时会自动注册进任务调度线程池。比如每 5 秒跑一次:

```
@Scheduled(fixedRate = 5000)
public void report() {
    System.out.println("执行中...");
}
```

2) 支持多种表达式类型, 最常用的是 `cron` 表达式, 比如每天凌晨 1 点执行:

```
@Scheduled(cron = "0 0 1 * * ?")
public void dailyTask() { ... }
```

也支持 `fixedDelay` (上一次执行完后隔多久)、`fixedRate` (不管上次是否完成, 固定频率触发)。

3) 默认是单线程串行执行所有任务, 如果某个任务耗时 10 秒, 而 `fixedRate` 是 5 秒, 那实际还是等上一个跑完才会触发下一个。想并行得自己配 `TaskScheduler` 或加 `@Async`。

4) 要启用这个功能, 主类得加上 `@EnableScheduling`。它底层依赖 Spring 的 `TaskScheduler` 抽象, 最终由 `ScheduledExecutorService` 驱动, 压根不经过外部调度系统。

注意分布式场景下别直接用这个做关键定时任务, 多实例会重复执行。真要分布式调度, 得上 `xxl-job` 或 `Elastic-Job` 这种带中心化调度和故障转移的框架。

## Spring 中的 @Cacheable 和 @CacheEvict 注解的作用是什么？

@Cacheable 和 @CacheEvict 是 Spring 基于 AOP 实现的声明式缓存控制手段，让你不用写重复的缓存读写代码。

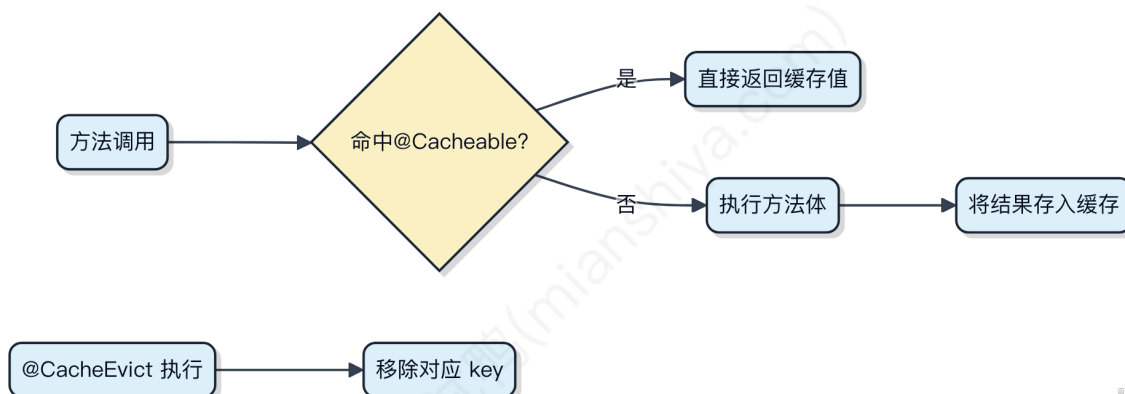
1) @Cacheable 用在方法上，第一次调用时执行方法体，把结果放进缓存。下次请求同样的参数，直接从缓存拿，**压根不经过方法体**。比如查用户信息，@Cacheable(value = "user", key = "#id") 就表示以 id 为 key 缓存返回值。

```
@Cacheable("books")
public Book getBookById(String isbn) {
    return findInDatabase(isbn);
}
```

2) @CacheEvict 用来清掉缓存，通常用在增删改操作后。比如更新一本书，就得把旧的缓存干掉，不然下次读到的就是脏数据。加 allEntries = true 能清整个缓存区，beforeInvocation = true 表示先清缓存再执行方法。

```
@CacheEvict(value = "books", key = "#isbn")
public void updateBook(String isbn, Book book) {
    saveToDatabase(book);
}
```

这套机制依赖 **CacheManager**，底层可以对接 Redis、Caffeine 等。注意 key 的生成策略，默认是用参数做 SimpleKey，复杂对象要确保 hashCode 和 equals 正确。如果方法抛异常，默认不会进缓存，可以用 unless 或 condition 控制更精细的行为。



## Spring 中的 @Conditional 注解的作用是什么？

@Conditional 是 Spring 里做条件化装配的开关。你给一个类或者配置加上它，Spring 就会根据你指定的条件来决定要不要把这个 Bean 拿出来创建。

它的核心是 **Condition 接口**，只要实现 matches 方法就行，返回 true 就加载，false 就跳过。比如你想在 Linux 系统下才启用某个服务，写个条件判断操作系统就行。

Spring Boot 里一堆衍生注解都是基于它的：1) @ConditionalOnClass: classpath 有这个类才生效 2)

@ConditionalOnMissingBean: 容器里没有这个 Bean 才创建 3) @ConditionalOnProperty: 配置文件里开了开关才起作用

举个例子，你用 Redis 做缓存，但想保留本地缓存作为 fallback。就可以写两个配置类，一个标上 `@ConditionalOnClass(Redis.class)`，另一个标 `@ConditionalOnMissingClass("redis.clients.jedis.Jedis")`，Spring 自动选合适的加载。

这样就不用硬编码逻辑，配置驱动就能切换行为，Starter 的自动装配全靠这套机制撑起来的。

## Spring 中的 @Lazy 注解的作用是什么？

`@Lazy` 注解控制的是 Bean 的初始化时机，简单说就是让 Bean 延迟到**第一次被使用时才创建**，而不是在应用启动时就一股脑全初始化好。

Spring 默认会把单例 Bean 在容器启动阶段就完成实例化和依赖注入，这叫预初始化。对于那些启动时用不到、或者初始化代价高的组件，这种做法其实浪费时间和资源。加上 `@Lazy`，就能把这部分“脏活累活”往后拖。

- 1) 加在配置类上，整个类里定义的 Bean 都延迟加载
- 2) 加在 `@Bean` 方法上，只对这个方法返回的 Bean 生效
- 3) 加在 `@Component` 类上，标记该组件为懒加载

比如你用 `@Service` 标记了一个报表服务，它依赖的数据源要查十几个表，启动时根本不会访问这个服务。这时候加上 `@Lazy`，能明显缩短启动时间。

有个细节要注意：如果 A 依赖 B，B 加了 `@Lazy`，但 A 没加，那 A 初始化时还是会去触发 B 的创建，相当于延迟失效。真正想延迟，得确保整条依赖链都支持懒加载。

代码上看，就是在声明时多一个注解：

```
@Lazy
@Bean
public ExpensiveService expensiveService() {
    return new ExpensiveService();
}
```

这种机制在大型项目里特别有用，像某些定时任务处理器、异步消息消费者，完全没必要在启动时就拉起来。用好 `@Lazy`，能让应用更快“扛住”启动过程。

## Spring 中的 @PropertySource 注解的作用是什么？

用来加载自定义的 Properties 配置文件，让里面的键值对能被 Spring 的 `Environment` 读取到。默认情况下 Spring 只会加载 `application.properties` 或 `application.yml`，如果你有额外的配置文件，比如 `jdbc.properties` 或 `merchant-config.properties`，就得靠它来引入。

- 1) 通过 `value` 属性指定文件路径，支持 `classpath:` 和 `file:` 协议
- 2) 可以配合 `@Value` 或 `Environment` API 取值
- 3) 如果文件不存在，默认会启动失败，加上 `ignoreResourceNotFound = true` 可以容忍

典型用法：

```
@PropertySource("classpath:jdbc.properties")
@Configuration
public class JdbcConfig {
    @Value("${db.url}")
}
```

```
private String dbUrl;
}
```

多个文件按顺序加载，后加载的会覆盖前面同名的 key。注意它只支持 `.properties` 文件，不支持 YAML。如果想加载 YAML，得自己实现 `PropertySourceFactory`。

实际项目里用得不多，多数人直接把所有配置扔 `application.yml`。但在模块化设计中，比如中间件或 SDK 需要自带独立配置时，这注解就很有用了，像早期的 Dubbo 就这么干过。

## Spring 中的 JPA 和 Hibernate 有什么区别？

JPA 是 Java 持久化规范，说白了就是一套接口定义，规定了 ORM 该有哪些功能，比如怎么映射实体、怎么查数据。Hibernate 则是这套规范的实现之一，真正干活的是它。

- 1) JPA 不绑定具体实现，你可以换别的 provider，比如 EclipseLink。但实际项目里，**Hibernate** 几乎成了事实标准，Spring Data JPA 默认用的也是它。
- 2) 你在代码里写 `@Entity`、`@Id` 这些注解，其实是 JPA 的，不是 Hibernate 自己的。Spring Data JPA 的 `JpaRepository` 接口也基于 JPA 规范设计，你面向的是接口编程。
- 3) 但一旦涉及高级功能，比如二级缓存、批量处理策略、自定义方言，就得用 Hibernate 特有的 API 了。这时候你会发现配置文件里还是得引入 Hibernate 的包，甚至写 `hibernate.hbm2ddl.auto` 这种属性。

举个例子，Spring Boot 配置：

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
    database: mysql
```

这里 `hibernate` 明明是实现，却出现在 JPA 配置下，就是因为 Spring 把两者融合得比较紧。

简单说：JPA 定规则，Hibernate 做脏活累活。你写业务用 JPA 注解和仓库接口，框架底层全交给 Hibernate 去扛住。

## Spring 中的 @EventListener 注解的作用是什么？

Spring 的 `@EventListener` 注解用来监听应用中发布的事件，相当于注册了一个回调函数。当某个事件被发布时，被注解的方法就会自动触发。

- 1) 方法上加上 `@EventListener`，Spring 就会在对应事件发生时调用它。比如用户注册成功后发一个 `UserRegisteredEvent`，你就可以写个监听方法发欢迎邮件。

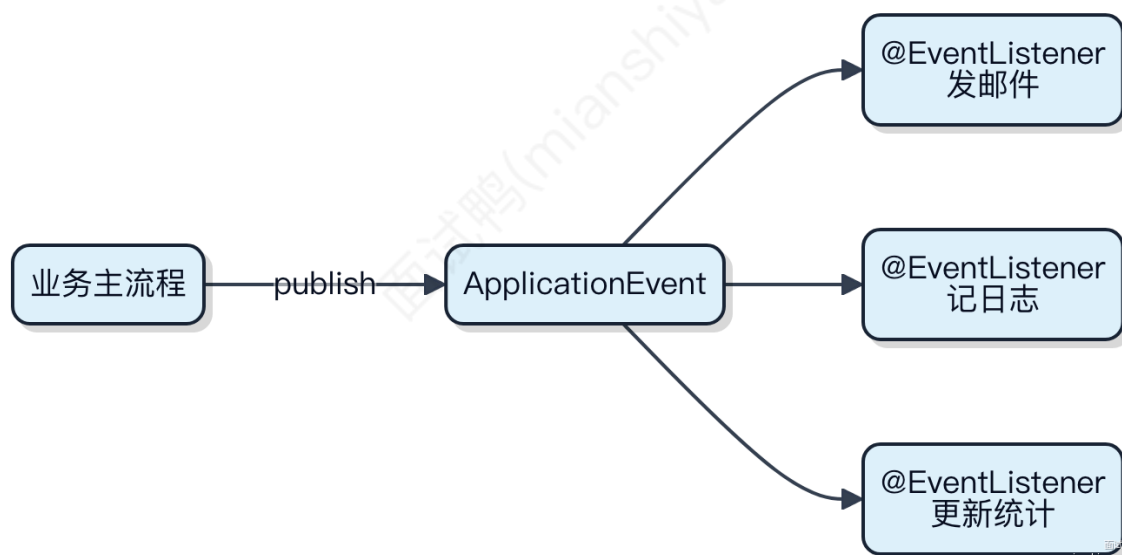
```
@EventListener
public void sendWelcomeEmail(UserRegisteredEvent event) {
    // 发送邮件逻辑
}
```

- 2) 监听的事件类型由方法参数决定。如果参数是 `ApplicationEvent` 子类，那这个方法就只响应这个类型的事件。支持泛型和条件表达式，比如用 `condition = "#event.userType == 'VIP'"` 控制只处理特定场景。

3) 默认是同步执行的，也就是说发布事件的地方会阻塞，直到所有监听器处理完。如果想异步执行，结合 `@Async` 一起用就行，但得提前启用 Spring 的异步支持。

4) 事件发布靠 `ApplicationEventPublisher`，一般通过注入它来发事件。整个机制基于观察者模式，典型的解耦手段，像 Spring Boot 的内置事件（如 `ContextRefreshedEvent`）也是这么玩的。

适合做业务逻辑解耦，比如订单创建后扣库存、更新积分这些不直接影响主流程的操作。不过别滥用，太多链式事件会让流程难以追踪，调试起来头疼。



### @Async 注解的原理是什么？

@Async 注解的底层其实是靠 Spring 的 AOP 拦截机制实现的。你加了 @Async，Spring 会在容器启动时扫描这些方法，然后通过代理对象把调用转发到线程池里执行。

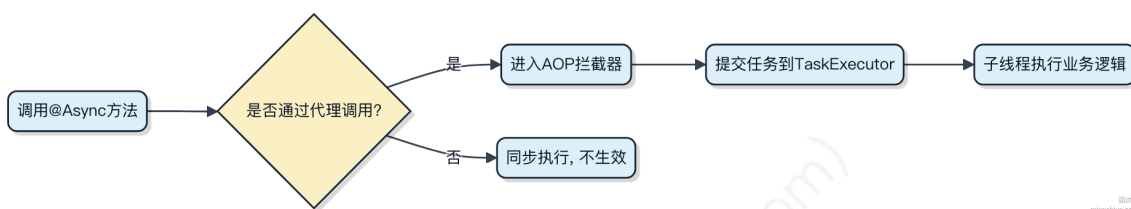
要让这个注解生效，得先在配置类上加上 @EnableAsync，不然压根不经过异步处理逻辑。Spring 会根据你的配置决定用哪种代理，比如 JDK 动态代理或者 CGLIB，这取决于目标类有没有实现接口。

方法被调用时，代理拦截器会从 TaskExecutor 中取一个线程来执行目标方法，原调用方直接返回，不会阻塞。默认情况下，Spring 用的是 SimpleAsyncTaskExecutor，但生产环境一般都会自定义线程池，比如配一个 ThreadPoolTaskExecutor，核心线程数设为 8，最大 20，队列容量 100 这种量级，避免资源耗尽。

注意一点，同一个类里方法直接调用，比如 methodA 直接调 methodB，即使 B 加了 @Async，也不会走代理，因为绕过了 AOP 拦截。这种情况得搞个 Service 自注入或者用 ApplicationContext 获取代理对象。

还有，被 @Async 标记的方法必须是 public 的，protected、private 或者包级可见都不行，这是由代理机制决定的。

返回值可以是 void 或者 Future 类型，比如 AsyncResult，方便后续做结果获取或异常处理。



## Spring 和 Spring MVC 的关系是什么?

Spring 是一个全能的 Java 企业级开发框架，它的目标是简化整个应用的开发。IoC 容器、AOP、数据访问、事务管理这些都归它管。你可以用它来写任何类型的 Java 应用，不局限于 Web。

Spring MVC 则是专门解决 Web 层问题的一个模块，它是 Spring 框架里的一个组件，全名叫 Spring Web MVC。你项目里要是想做基于 Servlet 的传统 Web 应用，比如前后端不分离的那种，那就会引入 `spring-webmvc` 这个包，它里面就包含了控制器、视图解析、前端控制器 `DispatcherServlet` 这一套东西。

你可以把 Spring 当成一个大平台，Spring MVC 就是插上面的一个 Web 插件。没有 Spring MVC，Spring 也能跑；但你要做 MVC 架构的 Web 应用，那就得靠它来处理 HTTP 请求的分发和响应流程。

- 1) Spring 提供基础能力：Bean 管理、依赖注入、切面编程
- 2) Spring MVC 在此基础上构建 Web 处理链：所有请求先打到 `DispatcherServlet`
- 3) 通过 `HandlerMapping` 找控制器，Adapter 调用方法，`ModelAndView` 渲染返回



注意，现在主流已经是 Spring Boot + RESTful API，Spring MVC 这套依然在底层支撑着 `@RestController` 的路由逻辑，只是封装得更透明了。

## Spring WebFlux 是什么？它与 Spring MVC 有何不同？

Spring WebFlux 是 Spring 5 引入的响应式编程框架，用来构建异步非阻塞的服务。它不是对 Spring MVC 的替代，而是多了一种选择，特别是在 I/O 密集或高并发场景下能更好地利用线程资源。

### 1) 运行模型不同

Spring MVC 基于 Servlet 容器，每个请求对应一个线程，阻塞时线程挂起，吞吐量受限于线程池大小。比如 Tomcat 默认最大线程数 200，超过就得排队。WebFlux 则基于 Reactor 模型，使用少量事件循环线程处理大量连接，数据流通过 **响应式流** (Reactive Streams) 传播，压根不经过传统同步 IO。

### 2) 编程范式不同

MVC 是命令式编程，代码一行行执行，调试直观。WebFlux 推荐函数式风格，用 `Mono` 和 `Flux` 表达异步数据流。比如返回单个结果用 `Mono<String>`，而不是 `String`。

```

@GetMapping("/async")
public Mono<String> getData() {
    return service.getDataAsync(); // 非阻塞调用
}
  
```

### 3) 适用场景有差异

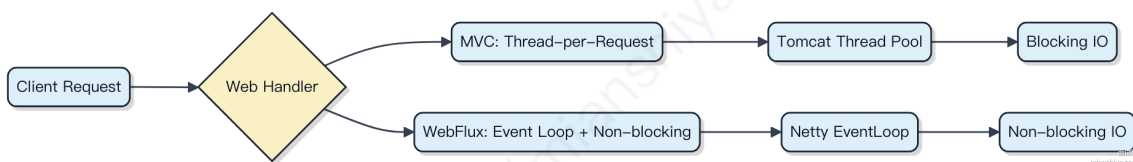
如果业务逻辑简单、依赖少，用 MVC 更稳更熟。但如果你在做网关类服务，比如集成 Spring Cloud Gateway，或者

需要长连接支持（如 WebSocket、SSE），WebFlux 能扛住更高并发。

#### 4) 容器支持也不同

MVC 可以跑在任何 Servlet 容器上。WebFlux 要么用 Netty，要么用支持异步 Servlet 3.1+ 的容器（如 Undertow），Tomcat 虽然也能跑，但优势不如在 Netty 上明显。

要不要上 WebFlux，关键看你的瓶颈是不是在 I/O。如果是 CPU 密集型，两者差别不大。而且响应式栈对数据库驱动也有要求，像 R2DBC 才能真正实现全链路异步，传统 JDBC 会把整个链路堵死。



### 介绍下 Spring MVC 的核心组件?

Spring MVC 的工作流程其实是个标准的请求分发处理模型，核心组件各司其职，把 HTTP 请求一步步转化成响应。

前端控制器 **DispatcherServlet** 是入口，所有请求都得经过它。它不干活，只负责调度，拿到请求后去找能处理它的“人”。

请求来了先看 **HandlerMapping**，这玩意就是个路由表，存着 URL 和对应处理器的映射关系。比如你访问 `/user`，它就知道该交给 **UserController** 的 `getUser` 方法处理。

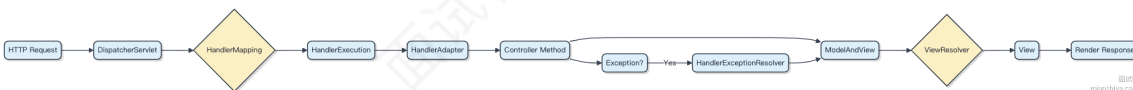
找到处理器后，由 **HandlerAdapter** 统一调用。它屏蔽了不同处理器类型的差异，不管是简单方法还是 **Controller** 接口实现，都能执行。

处理器执行完返回 **ModelAndView**，里面带着数据和视图名。接着 **ViewResolver** 上场，根据视图名找真正的视图实现，比如 JSP、Thymeleaf 模板。

最后 **View** 负责渲染，把模型数据填充进去生成 HTML，写回 **HttpServletResponse**。Model 数据则通过 request 域传递到页面。

异常也不放过，**HandlerExceptionResolver** 专门处理执行过程中的异常，可以返回错误页或 JSON 错误信息，保证流程不中断。

整个流程就像一条流水线，每个环节职责单一，扩展性强。像 `@RequestMapping`、`@Controller` 注解背后都是这套机制在支撑。



### 什么是 Restful 风格的接口?

REST 是一种设计 Web API 的架构风格，不是强制标准，核心是把服务器上的资源映射成 URL，用 HTTP 动词来操作这些资源。

比如一个用户资源 `/users`，增删改查就对应：

- `GET /users` 获取列表



- `POST /users` 创建新用户
- `GET /users/123` 获取 ID 为 123 的用户
- `PUT /users/123` 更新整个用户
- `PATCH /users/123` 部分更新用户
- `DELETE /users/123` 删除用户

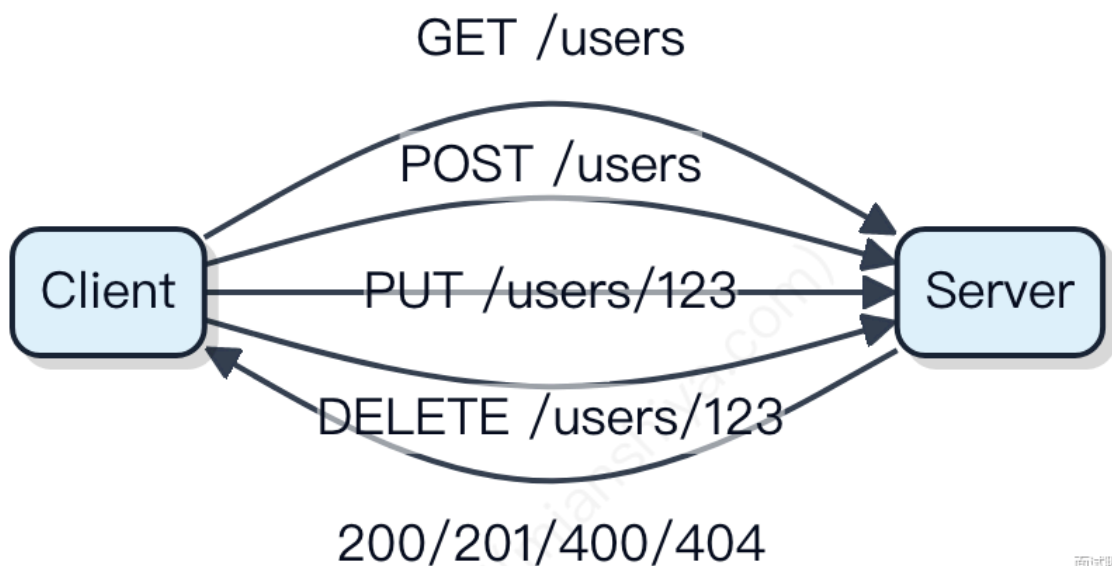
状态码也得用对，成功返回 200（或 201），客户端错误给 400，找不到资源是 404，服务器异常才扔 500。这样调用方能靠标准机制理解结果。

URI 设计别带动词，别写成 `/getUserById?id=123`，这是早期 RPC 风格的写法。REST 要的是名词 + HTTP 方法组合表达意图。

数据格式通常用 JSON，请求和响应都走它，前后端约定好结构就行。像 Spring Boot 默认就支持这种模式，加个 `@RestController`，方法上配 `@GetMapping` 就完事。

版本控制一般放 URL 或 Header 里，比如 `/v1/users`，避免升级搞挂老客户端。

- 1) 资源导向：一切是资源，URL 是资源地址
- 2) 无状态：每次请求包含全部信息，服务端不保存上下文
- 3) 统一接口：HTTP 方法 + 状态码 + 标准化 URI



面试题  
mianshiya.com

### Spring MVC中的Controller是什么？如何定义一个Controller？

Spring MVC里的Controller本质是处理HTTP请求的入口，它负责接收前端发来的请求，执行业务逻辑，然后返回响应结果。你不需要关心底层Socket怎么读数据，框架已经把请求解析好了，你只用关注“这个请求要干什么”。

定义一个Controller最常见的方式是加 `@Controller` 注解，再配合 `@RequestMapping` 指定路径。比如：

```

@Controller
@RequestMapping("/user")
public class UserController {
    @GetMapping("/{id}")
    
```



```
public ResponseEntity<User> getUser(@PathVariable Long id) {  
    User user = userService.findById(id);  
    return ResponseEntity.ok(user);  
}  
}
```

如果你返回的是JSON这类数据而不是页面，可以直接用 `@RestController`，它等于 `@Controller + @ResponseBody`，所有方法默认返回数据到响应体。

- 1) 类上标注 `@Controller` 或 `@RestController`
- 2) 用 `@RequestMapping` 及其变体（如 `@GetMapping`、`@PostMapping`）声明请求映射
- 3) 方法参数通过 `@RequestParam`、`@PathVariable`、`@RequestBody` 绑定请求数据

注意别忘了组件扫描，确保Spring能发现这个类，一般启动类加上 `@SpringBootApplication` 就够了，它包含了 `@ComponentScan`。

请求进来时，`DispatcherServlet` 会根据路径找到对应的 Controller 方法，通过反射调用，参数和返回值由 `HandlerMethodArgumentResolver` 和 `MessageConverter` 自动处理，整个过程对开发者透明。

## Spring MVC 中的视图解析器有什么作用？

视图解析器在 Spring MVC 里干的活很明确：把控制器返回的逻辑视图名，转成实际能用的视图资源路径。

比如你写个 Controller 返回 `"user/list"`，这只是一个逻辑名字。真正的页面可能在 `WEB-INF/views/user/list.jsp`，也可能是个 Thymeleaf 模板在 `templates/user/list.html`。这个“翻译”工作就是视图解析器做的。

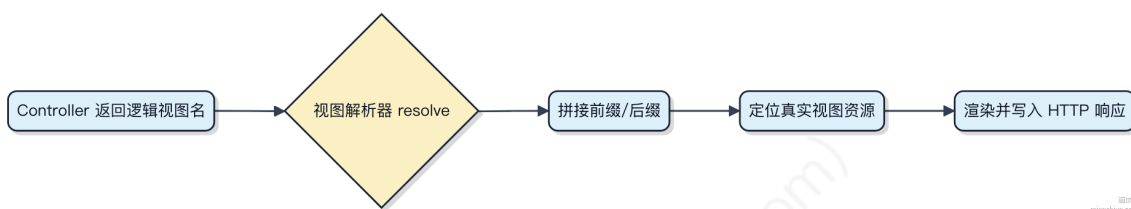
最常见的实现是 `InternalResourceViewResolver`，你一般会配置一个前缀和后缀：

```
@Bean  
public ViewResolver viewResolver() {  
    InternalResourceViewResolver resolver = new InternalResourceViewResolver();  
    resolver.setPrefix("/WEB-INF/views/");  
    resolver.setSuffix(".jsp");  
    return resolver;  
}
```

这样 `user/list` 就自动拼成 `/WEB-INF/views/user/list.jsp`。整个过程对开发者透明，你只管返回逻辑名就行。

另外像 Thymeleaf、FreeMarker 这些模板引擎，也有对应的视图解析器，它们能处理更复杂的视图逻辑，比如片段渲染、国际化布局等。

如果项目前后端分离，Controller 直接返回 JSON，那视图解析器压根不经过，靠 `@ResponseBody` 或 `RestController` 就直接把数据写回响应体了。



## Spring 中的 ApplicationContext 是什么？

ApplicationContext 是 Spring 容器的核心接口之一，它在 BeanFactory 基础上扩展了更多企业级功能。你可以把它看作一个“高级容器”，不只是能装 bean，还能支持国际化、事件发布、资源加载等。

它启动时会**预加载所有单例 bean**，这样应用一启动就能发现配置错误，而不是运行到一半才出问题。这一点和懒加载的 BeanFactory 不一样，实际项目里基本都用 ApplicationContext。

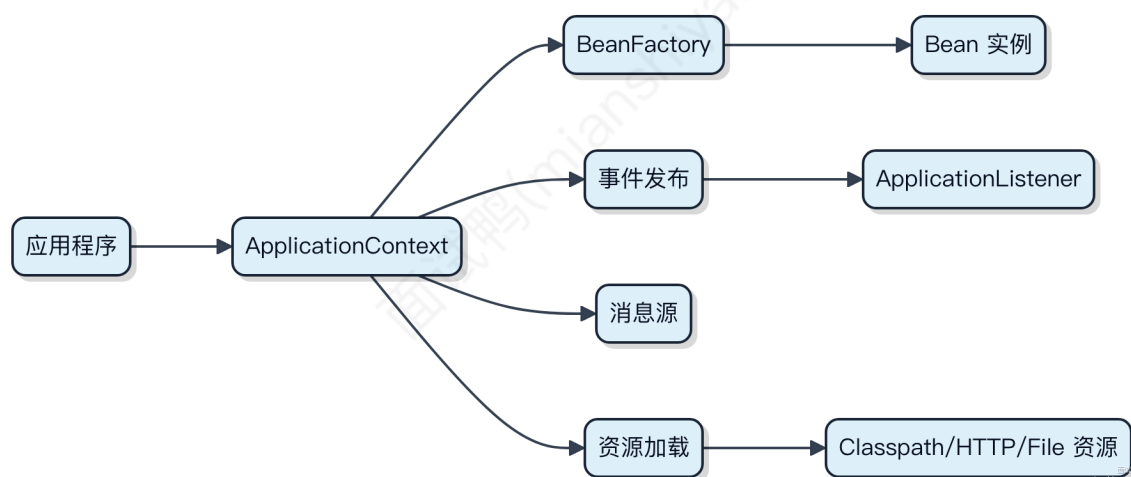
常见的实现类有几个：1) ClassPathXmlApplicationContext 从 classpath 下加载 XML 配置 2) FileSystemXmlApplicationContext 从文件系统读配置 3) AnnotationConfigApplicationContext 支持注解配置，比如扫描 @Configuration 类

代码上通常这么用：

```
ApplicationContext ctx = new AnnotationConfigApplicationContext(AppConfig.class);
MyService service = ctx.getBean(MyService.class);
```

它还支持**事件监听机制**，比如你发个 ApplicationEvent，监听器就能收到。Spring Boot 的很多自动装配就是基于这个做的。

另外像 AOP、事务管理这些功能，也都依赖 ApplicationContext 提供的生命周期回调和后置处理器机制。可以说，整个 Spring 生态的扩展能力，都是搭在这个容器上的。



## 为什么 Spring 循环依赖需要三级缓存，二级不够吗？

Spring 解决循环依赖靠的是提前暴露对象的机制，而三级缓存的设计是为了在**不破坏 Bean 生命周期的前提下**，拿到**正确的代理对象**。

- 1) 一级缓存是 `singletonObjects`，存完全初始化好的单例 Bean。
- 2) 二级缓存是 `earlySingletonObjects`，存提前曝光的原始对象（还未完成属性填充和初始化）。
- 3) 三级缓存是 `singletonFactories`，存能创建早期引用的 `ObjectFactory`，用来生成可能需要被代理的对象。

关键点在于：如果两个 Bean 都涉及 AOP 代理，比如都加了 `@Transactional`，那么它们相互注入时必须都是代理对象，而不是原始实例。

假设只有二级缓存：

- A 创建过程中发现自己被 B 引用，就把原始实例放二级缓存。
- B 去拿 A 的时候，只能拿到原始对象，后续再对 A 进行代理，那 B 拿到的就是错的，压根不经过代理逻辑。

有了三级缓存就不一样了：

- A 创建时，把自己包装成一个 `ObjectFactory` 放进三级缓存。
- B 需要 A 时，通过这个工厂触发 `getEarlyBeanReference`，拿到的是经过所有 `BeanPostProcessor` 处理后的早期引用——也就是正确的代理对象。
- 等 A 最终创建完，还会把最终版本升级到一级缓存，保证一致性。



所以，三级缓存的核心价值就是：**延迟代理对象的生成时机，同时确保中间环节拿到的是正确增强过的早期引用。**

## Spring 如何解决循环依赖？

Spring 解决循环依赖主要靠三级缓存和提前暴露对象引用的机制，核心是把实例化和初始化分开。

- 1) 当 A 依赖 B，B 又依赖 A 时，Spring 在创建 A 的过程中，new 出 A 实例后就马上把它放进**早期暴露缓存** (`singletonFactories`)，这个工厂能返回尚未完全初始化的 A 对象。
- 2) 接着去创建 B，发现需要 A，就会从 `singletonFactories` 拿到那个“半成品”A 实例。此时 B 能完成注入，然后 B 初始化完毕放入一级缓存（单例池）。
- 3) 再回到 A 的初始化，从一级缓存拿到完整的 B，继续完成 A 的属性填充和初始化，最后 A 也进一级缓存，整个流程闭环。

这里的关键是三个缓存层级：

- 一级：`singletonObjects`，放完全初始化好的 Bean
- 二级：`earlySingletonObjects`，放提前暴露的原始对象（用于解决普通循环依赖）
- 三级：`singletonFactories`，存放 `ObjectFactory`，能生成早期引用

但要注意，这种机制只能解决**单例作用域下的 setter 注入循环依赖**。构造器注入的循环依赖 Spring 压根不经过缓存，直接抛异常。还有原型（prototype）作用域也不支持，每次都是新对象，没法缓存。

代码层面不需要额外操作，只要用 `@Autowired` 注入就行，框架自动处理。



## 什么是循环依赖（常问）？

Spring 里的循环依赖，指的是两个或多个 Bean 互相依赖，形成闭环。比如 A 依赖 B，B 又依赖 A，初始化时就可能卡住。

Spring 能解决**部分**循环依赖，靠的是三级缓存。创建中的 Bean 会提前暴露到缓存里，让对方能引用到一个“半成品”实例。

1) 一级缓存是 `singletonObjects`，存完全初始化好的单例。2) 二级缓存是 `earlySingletonObjects`，存提前暴露的原始对象（还没填充属性完）。3) 三级缓存是 `singletonFactories`，存能生成这个早期对象的工厂函数。

只有**单例 Bean**且通过 **setter 或字段注入**，Spring 才能自动解环。构造器注入的循环依赖，压根不经过缓存，直接抛异常。



原型 (prototype) Bean 不走缓存，每次都要新创建，遇到循环依赖直接报错。所以设计时要警惕这种隐式耦合，必要时用 `@Lazy` 推迟加载。

## Spring AOP 和 AspectJ 有什么区别？

Spring AOP 和 AspectJ 都解决面向切面编程的问题，但实现方式和能力差距挺大。

Spring AOP 是运行时织入，基于动态代理。如果目标对象实现了接口，就用 JDK 动态代理；没有接口就用 CGLIB 生成子类。切面逻辑在方法调用时才被拦截，**织入**发生在运行期。它只能拦截 Spring 容器管理的 bean 的方法执行，功能相对有限，但集成简单，适合大多数业务场景的增强，比如日志、事务。

AspectJ 是编译时织入（也可以在类加载时），直接修改字节码。它通过 `ajc` 编译器或者 LTW (Load Time Weaving) 把切面代码“插”进目标类里，性能更高，支持的方法切入点也更细，比如构造器、字段访问都能监听。你写个 `private` 方法，它也能拦住，Spring AOP 压根不经过这种地方。

- 1) Spring AOP 只支持方法级别的 `execution` 切入点
- 2) AspectJ 支持 `call`、`get`、`set`、`handler` 等多种 `join point`
- 3) Spring AOP 天然和 IoC 容器集成，声明式配置方便
- 4) AspectJ 需要额外工具链，学习成本高，但能力更强

代码上，Spring AOP 用 `@Aspect + @Around` 就行，而 AspectJ 要么用 `.aj` 文件，要么配合 LTW 配置织入器。

一般业务开发用 Spring AOP 足够了，像监控、埋点这种需要深度拦截的场景才会考虑 AspectJ。真要上 AspectJ，记得压测对比下启动时间和内存开销。

## 能说说 Spring 拦截链的实现吗？

Spring 拦截链的实现其实靠的是 AOP 里的责任链模式，把一堆 `HandlerInterceptor` 按顺序串起来，在请求处理的不同阶段挨个执行。

这些拦截器在 `DispatcherServlet` 处理请求时被触发。整个流程分三步：`preHandle`、`postHandle`、`afterCompletion`。`preHandle` 在控制器方法执行前调用，返回 `false` 直接中断请求，后面的拦截器和 controller 都不会执行。`postHandle` 在 controller 执行完但还没渲染视图时运行。`afterCompletion` 是最后收尾，不管成功失败都会走，适合做资源清理。

- 1) `preHandle` 执行顺序是从前往后，按注册顺序依次调
- 2) `postHandle` 和 `afterCompletion` 是从后往前，类似栈结构，保证释放顺序正确

比如你用 Spring Boot 配了个 `LoggingInterceptor` 和 `AuthInterceptor`，请求进来先过 `preHandle` 链，如果某个环节鉴权失败，后续直接断掉，响应就回去了。而日志的 `afterCompletion` 能确保即使抛异常也能记录耗时。

代码上关键就是实现 `HandlerInterceptor` 接口：

```
public class AuthInterceptor implements HandlerInterceptor {
    public boolean preHandle(HttpServletRequest req, HttpServletResponse resp,
Object handler) {
        if (noAuth(req)) {
            resp.setStatus(401);
            return false;
        }
        return true;
    }
}
```

注册的时候通过 `WebMvcConfigurer` 加进去就行，顺序由注册顺序决定。注意它和 `Filter` 不一样，`Filter` 是 `Servlet` 容器级别的，**拦截链**是 `Spring MVC` 自己维护的，压根不经过 `Filter` 那层。常见像日志、权限、性能监控都用它做，脏活累活全包了。

### Spring AOP默认用的是什么动态代理，两者的区别？

`Spring AOP` 在生成代理对象时，会根据目标类是否实现接口来决定使用哪种动态代理。如果目标类实现了至少一个接口，`Spring` 就用 **JDK 动态代理**，所有接口方法调用都会被拦截。要是没实现接口，那就上 **CGLIB**。

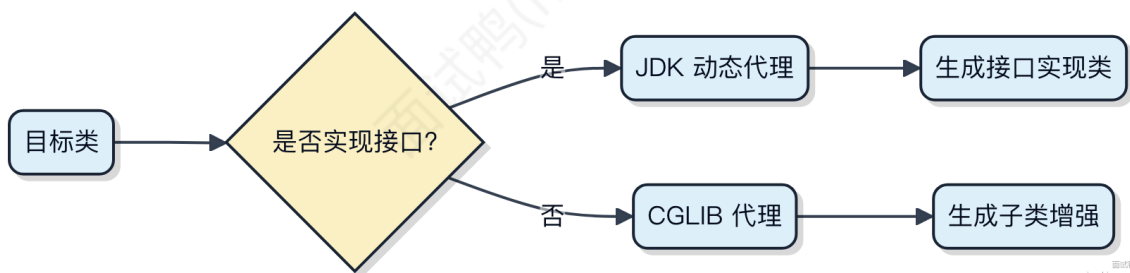
`JDK` 动态代理是 `Java` 原生支持的，核心是 `InvocationHandler` 和 `Proxy`，它要求被代理类必须有接口，生成的代理对象是接口的实现类实例。这种方式性能不错，而且不依赖第三方库。

`CGLIB` 是基于 `ASM` 字节码框架实现的，它通过继承的方式创建子类来增强功能，所以目标类不能是 `final` 的，方法也不能是 `final` 或 `private` 的。它没有接口也能代理，适用性更广，但会多一层继承关系，启动时要生成 `class` 文件，稍微耗点内存。

举个例子，你写了个 `UserService` 实现了 `UserOperations` 接口，`Spring` 默认走 `JDK` 代理。但如果你的类是 `package-private` 或者压根没定义接口，那只能靠 `CGLIB` 来搞定了。

实际项目里像 `Spring Boot` 自动配置、事务管理这些场景，底层都可能混用这两种方式，具体看你的类结构。

要不要切换？可以强制用 `CGLIB`，只要在配置类上加 `@EnableAspectJAutoProxy(proxyTargetClass = true)` 就行。不过一般没必要，`Spring` 自己能选对。



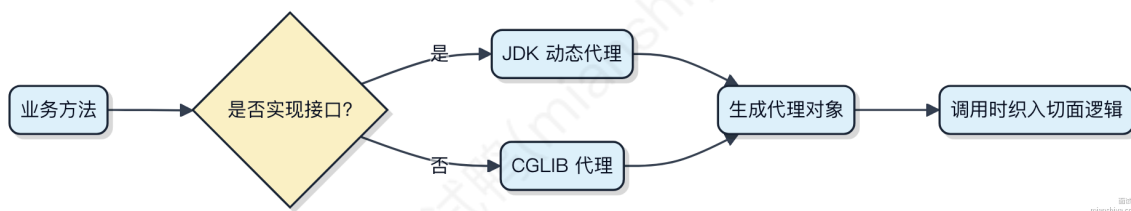
### 什么是 AOP?

AOP，也就是面向切面编程，本质是为了解决横切关注点的代码复用和集中管理。像日志记录、权限校验、事务管理这些功能，散落在各个业务方法里，改起来费劲还容易漏。AOP 就是把这些**横切逻辑**抽出来，统一织入到目标方法的

特定位置。

Spring AOP 的实现依赖动态代理。如果目标类实现了接口，默认用 JDK 动态代理；没有接口就用 CGLIB。整个过程不需要修改原有代码，通过配置或注解就能把切面加进去。比如你用 `@Around` 写个环绕通知，就能在方法执行前后插入逻辑。

- 1) 切点 (Pointcut) 决定哪些方法要被拦截，通常用表达式匹配方法签名
- 2) 通知 (Advice) 定义在切点的哪个时机执行，比如前置、后置、环绕
- 3) 切面 (Aspect) 是切点和通知的组合，描述“在哪执行”和“执行什么”



典型场景就是 Controller 层统一打日志，或者 Service 方法上加 `@Transactional`。但别滥用，像业务逻辑本身就不该塞进切面里，否则别人看不懂流程。简单说，AOP 是工具，不是魔法，用好了事半功倍，用歪了就是坑。

## Spring 一共有几种注入方式？

Spring 的依赖注入方式主要就三种：构造器注入、Setter 注入、字段注入。实际项目里最常见的是前两种。

- 1) 构造器注入是现在推荐的方式，尤其在类的依赖不可变时。它能保证依赖不为空，也方便写单元测试。Lombok 的 `@RequiredArgsConstructor` 能帮你省掉大量构造函数代码。

```
@Service
public class OrderService {
    private final PaymentService paymentService;

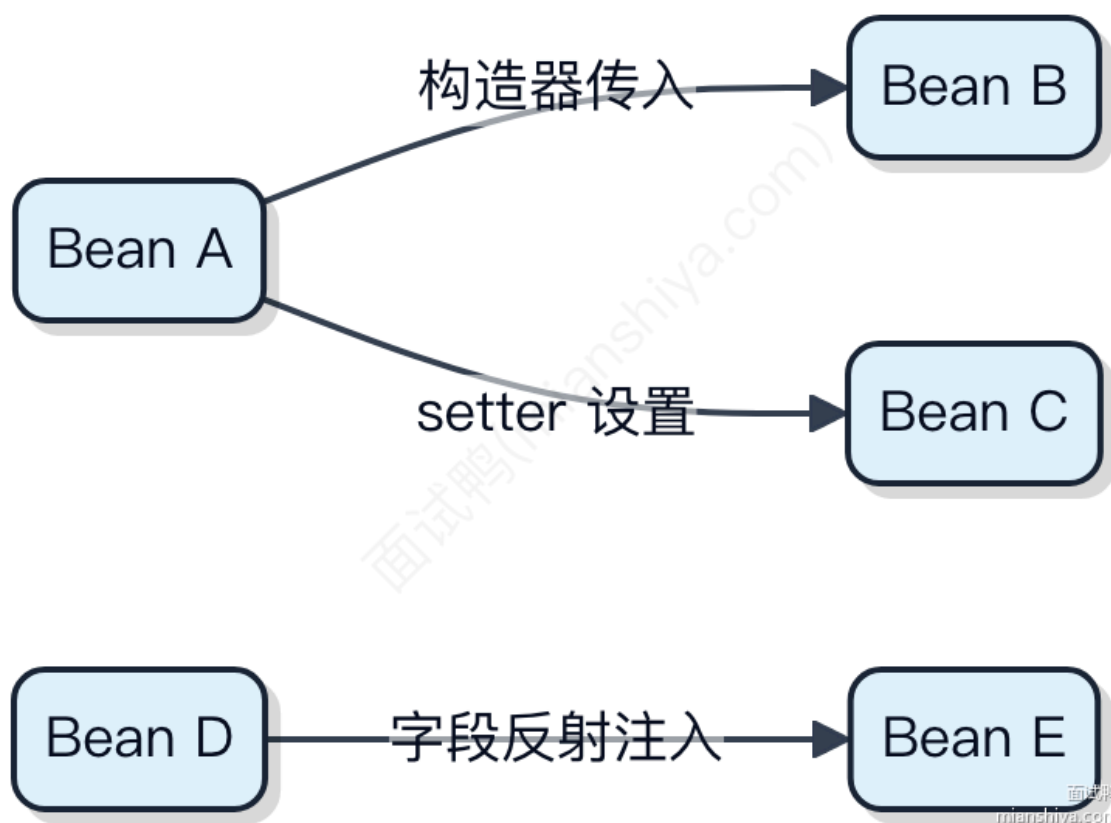
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

- 2) Setter 注入灵活性高，适合可选依赖。但对象可能处于不完整状态，而且 setter 方法对外暴露，破坏封装性。

- 3) 字段注入写起来最简单，直接 `@Autowired` 加在字段上。但它把依赖隐藏了，不利于测试，也不符合“显式依赖”原则。阿里规约明确禁止这种写法。

其实从 Spring 4.3 开始，如果类只有一个构造函数，连 `@Autowired` 都可以省略，容器会自动装配。这进一步推动大家用构造器注入。

要不要用字段注入？小工具类或者 demo 无所谓，但正式项目建议别图省事。长期来看，构造器注入更安全，代码也更干净。



图里字段注入被标红了，就是提醒你——能不用就不用。

### 说下 Spring Bean 的生命周期？

Spring Bean 的生命周期从创建到销毁，整个过程由容器全权管理，我们可以在关键节点插入自定义逻辑。

- 1) 实例化 Bean：容器通过反射调用构造方法创建对象，这时候还没进行属性注入。
- 2) 属性填充：依赖的字段通过 setter 或自动注入方式赋值，比如 @Autowired 标注的成员会被处理。
- 3) 初始化前处理：如果类实现了 BeanPostProcessor，它的 postProcessBeforeInitialization 方法会执行，这是个扩展点，像 AOP 代理就可能在这里生成。
- 4) 初始化操作：先看有没有实现 InitializingBean 接口，有的话调用 afterPropertiesSet；然后检查是否配置了 init-method，比如 XML 中指定或 @Bean(initMethod="xxx")，都会在这个阶段触发。
- 5) 初始化后处理：BeanPostProcessor 的 postProcessAfterInitialization 再次介入，很多代理逻辑都在这步完成，比如事务代理。
- 6) 放入单例池：如果是单例，默认会把最终对象放进一级缓存 singletonObjects，后续 getBean 直接返回。
- 7) 使用阶段：Bean 被应用程序正常使用，比如作为 Controller 处理请求，或被其他 Bean 引用。
- 8) 销毁阶段：容器关闭时，先调用 DisposableBean 的 destroy 方法，再执行 destroy-method 指定的方法，比如释放数据库连接、关闭线程池。





prototype 类型的 Bean 容器不负责销毁，不会调用销毁方法。这个流程里最常被利用的是两个 **BeanPostProcessor** 扩展点，Spring 的各种功能都靠它织入。

## Spring 中的 ObjectFactory 是什么？

Spring 的 **ObjectFactory** 是个简单的工厂接口，核心就一个 `getObject()` 方法，用来延迟获取 Bean 实例。你不用一口气把所有 Bean 都初始化好，等真正用到的时候再通过它去拿，省资源。

这玩意儿最常见的场景就是解决循环依赖和原型 Bean 注入到单例里的问题。比如你有个单例 Service，里面注入了一个 prototype 作用域的 Mapper，每次调用都得是新对象。如果直接注入，Spring 在启动时就给你绑死一个实例了，压根不经过作用域控制。这时候用 **ObjectFactory** 包一层，每次调用 `getObject()` 才真正触发创建，就能拿到新的 prototype Bean。

它也是 **ObjectProvider** 的底层支撑，后者是前者的增强版，支持默认值、可选注入这些更友好的 API。像在 `@Autowired` 里用 **ObjectProvider** 做集合注入或者容错处理，背后其实就是 **ObjectFactory** 在干活。

代码上很简单：

```
@Autowired
private ObjectFactory<MyService> myServiceFactory;

public void useService() {
    MyService service = myServiceFactory.getObject();
    service.doSomething();
}
```

- 1) 每次调 `getObject()` 都会触发 Bean 的创建流程
- 2) 适用于需要动态获取、避免提前初始化的场景
- 3) 和作用域管理强相关，尤其是 prototype 和 request 这种非单例

## Spring 中的 FactoryBean 是什么？

**FactoryBean** 是 Spring 里一个特殊的接口，它不是用来直接注册普通 Bean 的，而是让你能更精细地控制某个 Bean 的创建过程。你实现这个接口后，Spring 容器会调用它的 `getObject()` 方法来获取真正的实例，而不是直接用构造器创建。

- 1) 当你在 IOC 容器里通过 `getBean("xxx")` 拿对象时，如果这个 name 对应的是个 **FactoryBean**，拿到的是它生产的对象。想拿 **FactoryBean** 本身得加个 `&` 前缀，比如 `getBean("&dataSource")`。
- 2) 这种机制在整合第三方库时特别常见。像 MyBatis 的 **SqlSessionFactoryBean** 就是典型例子，你没法靠简单的 `new` 来创建一个可用的 **SqlSessionFactory**，得先解析 Mapper 文件、设置数据源等等，这些脏活累活都封装在 `getObject()` 里了。
- 3) 它和 `@Bean` 注解不一样。`@Bean` 是配置类里写逻辑返回对象，而 **FactoryBean** 本身是个可复用的工厂实现，适合模板化流程。

```
public class DataSourceFactoryBean implements FactoryBean<DataSource> {
    public DataSource getObject() throws Exception {
```

```

        return createPooledDataSource(); // 复杂创建逻辑
    }

    public Class<?> getObjectType() {
        return DataSource.class;
    }
}

```

这种方式让 Spring 能管理那些不能直接反射生成的复杂对象，扩展性一下子打开了。

## Spring 中的 BeanFactory 是什么？

BeanFactory 是 Spring 框架最基础的 IoC 容器接口，负责管理 Bean 的生命周期和依赖注入。它用延迟初始化的方式创建 Bean，只有在真正获取时才触发实例化。

- 1) 它定义了 `getBean` 这样的核心方法，通过名称或类型从容器中获取 Bean 实例。比如你调用 `context.getBean("userService")`，底层就是 BeanFactory 在干活。
- 2) 实际开发中我们更多用 `ApplicationContext`，它是 BeanFactory 的子接口，提供了更多企业级功能，比如事件发布、国际化、AOP 支持等。但归根结底，`ApplicationContext` 内部还是组合了一个 BeanFactory 来做 Bean 的管理。
- 3) BeanFactory 不会主动去解析配置或扫描类路径，这部分工作由具体的实现类完成。比如 `XmlBeanFactory` 会读取 XML 配置，而 `DefaultListableBeanFactory` 是最常用的完整实现，Spring Boot 自动装配也基于它扩展。

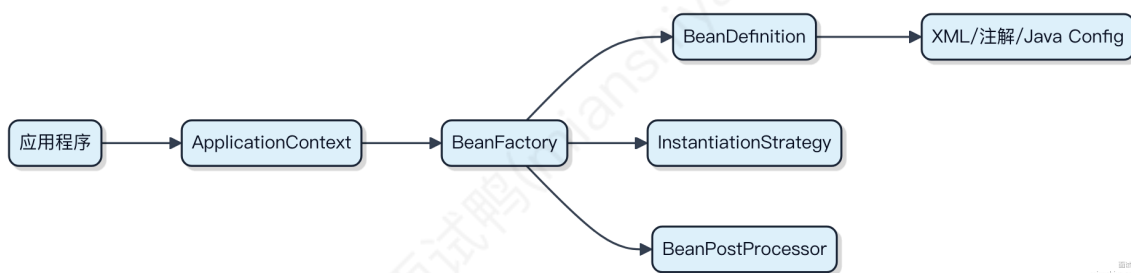
代码上，你可以这么理解：

```

BeanFactory factory = new XmlBeanFactory(new ClassPathResource("beans.xml"));
UserService userService = (UserService) factory.getBean("userService");

```

这个过程不经过任何代理或增强，纯粹是根据定义创建对象。像 `@Autowired`、`@PostConstruct` 这些注解的支持，其实是后置处理器（`BeanPostProcessor`）做的，它们注册到 BeanFactory 中，在 Bean 创建过程中被回调执行。



简单说，**BeanFactory** 就是 Spring 管理对象的“内核”，其他都是围绕它构建的封装和扩展。

## 什么是 Spring Bean？

Spring Bean 是被 Spring 容器管理的对象实例。你写一个类，比如 `UserService`，容器负责创建它、组装它依赖的其他组件（比如 `UserDao`），再交给需要的地方使用。

- 1) 生命周期由容器控制。从创建、初始化回调（`@PostConstruct`）、属性填充，到销毁前回调（`@PreDestroy`），整个过程都由 Spring 接管。你可以通过实现 `InitializingBean` 或 `DisposableBean` 接口，或者用注解来定义初始化和销毁逻辑。

2) 配置方式灵活。可以用 XML 声明 `<bean class="..." />`，也可以用 `@Component`、`@Service` 这类注解配合 `@ComponentScan` 自动注册。Java Config 方式更主流，比如：

```
@Configuration
public class AppConfig {
    @Bean
    public UserService userService() {
        return new UserService(userDao());
    }
}
```

3) 作用域不只有单例。虽然默认是 `singleton`，但也能设为 `prototype`（每次获取都新建）、`request`、`session` 等，适合 Web 场景。

Bean 和普通对象的区别在于是否被 IoC 容器接管。`new` 出来的对象再怎么管，也不是 Bean；而一个简单的 POJO，只要被容器实例化和管理，就是 Bean。

别把 Bean 想得太复杂，它其实就是 Spring 帮你“**new**”出来，并且“**管到底**”的对象。脏活累活都交给容器，你要做的就是声明规则。

## Spring 中的 DI 是什么？

依赖注入（DI）是 Spring 框架最核心的能力之一，它让对象之间的依赖关系由容器来管理，而不是在代码里硬编码。

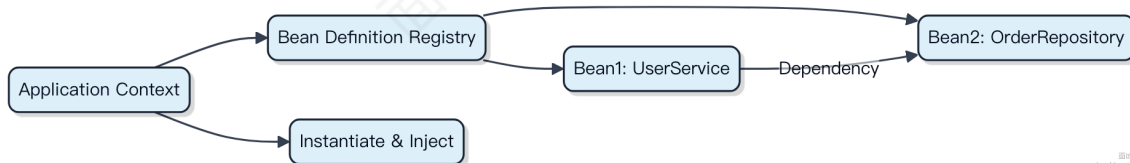
比如一个 Service 需要用到 Repository，传统做法是在 Service 里 `new` 一个 Repository 实例，耦合度高，测试困难。DI 的做法是，Spring 容器在启动时根据配置或注解把 Repository 实例“塞”进 Service 里，运行时直接用就行。

最常见的注入方式有三种：

- 1) 通过构造函数注入，推荐这种方式，不可变且便于单元测试
- 2) 通过 setter 方法注入，灵活性高但对象可能处于不完整状态
- 3) 通过字段直接注入（`@Autowired` 注解字段），写起来最简单，但降低了可测试性和灵活性，不推荐在复杂项目中使用

Spring 在创建 Bean 时会解析依赖关系图，按需实例化并完成注入。整个过程基于反射和 JavaBean 规范实现，支持循环依赖的处理（借助三级缓存）。

举个例子，你用 `@Component` 标记组件，`@Autowired` 标记要注入的字段或构造函数，Spring 启动时自动完成装配。像 MyBatis 的 `SqlSessionFactory`、Redis 的 `RedisTemplate`，都是通过 DI 管理的典型场景。



本质上，DI 是控制反转（IoC）思想的具体实现，把对象的创建和组装责任从应用代码转移到容器，专注业务逻辑即可。

## Spring IOC 有什么好处？

Spring IOC 的本质是把对象的创建和依赖管理交给容器来处理，开发者不再需要手动 new 对象或者硬编码依赖关系。

- 1) 控制反转最直接的好处是解耦。比如一个 Service 依赖 Dao，以前要在代码里写 `new UserDao()`，现在通过配置或注解声明依赖，运行时由容器注入。换个实现类都不用改代码，只需要改配置。
- 2) 统一管理对象生命周期。像单例、原型这些作用域，容器都帮你管好了。比如 Web 环境下常见的单例 Bean，容器保证整个应用只存在一个实例，省得自己搞静态变量或者双重检查锁那一套。
- 3) 支持延迟初始化。有些 Bean 可以设置 `lazy-init`，真正用到才创建，启动速度快不少。特别是那些大项目，几百个 Bean 全在启动时加载，根本扛不住。
- 4) 配合 AOP 做增强也更方便。因为所有 Bean 都归容器管，你要加事务、日志、缓存，直接在容器层面织入就行，不用侵入业务代码。

```
@Service
public class UserService {
    @Autowired
    private UserDao userDao; // 容器自动注入，压根不经过开发者手动创建
}
```

这种模式在大型系统里特别关键。像微服务架构中，用 Spring Cloud 搭的服务，Bean 之间跨模块调用频繁，靠手动维护依赖根本搞不定。IOC 让整个结构变得灵活又可控。

## 什么是 Spring IOC?

Spring IOC 解决的是对象之间的依赖关系管理问题。我们不再用 `new` 去创建对象，而是把对象的控制权交给容器去管理，这就是“控制反转”——原来由程序自己控制的东西，现在反转给了外部容器。

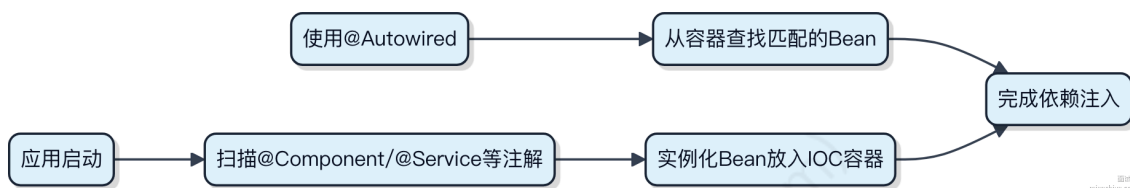
容器启动时会读取配置（比如 XML 或注解），把一个个对象实例化好，放到 **IOC 容器**里。需要用的时候，直接从容器拿，不需要自己 `new`。对象之间的依赖关系，比如 Service 需要 Dao，容器会自动注入进去，这个过程叫依赖注入（DI），是 IOC 的实现方式。

举个例子，你写了个 `UserService` 上面加了 `@Service`，`UserDao` 加了 `@Repository`，Spring 启动时就把它们都实例化，当你在 `UserController` 里用 `@Autowired` 注入 `UserService`，容器会直接把准备好的实例塞进去。

```
@Service
public class UserService {
    @Autowired
    private UserDao userDao;
}
```

常见场景像 Bean 的作用域控制，单例默认就是容器里只有一份。如果是 Web 应用，结合 `@Controller`、`@Component` 这些注解，整套对象体系都能被自动装配起来，省得手动维护对象生命周期。

IOC 让代码更松耦合，测试也方便，Mock 对象可以直接塞进容器替换真实实现。但过度依赖注解和隐式注入的话，排查问题会有点麻烦，尤其是 Bean 冲突或者循环依赖的时候。



## Spring 的单例 Bean 是否有并发安全问题?

单例 Bean 有没有并发安全问题，关键看它有没有状态。Spring 的单例指的是容器里一个类只有一个实例，这个实例会被多个线程共享。

如果这个 Bean 是无状态的，比如它只有方法，没有可变的成员变量，那压根不经过实例变量，天然就是线程安全的。大多数业务 Service、Controller 都是这种，不用操心。

但一旦你在单例 Bean 里加了可变的成员变量，比如 `int count` 或者一个 `List`，多个线程同时改这个值，问题就来了。这时候没有额外同步措施，肯定出问题。

举个例子：

```
@Service
public class CounterService {
    private int count = 0; // 可变状态

    public void increment() {
        count++; // 非原子操作，多线程下会丢更新
    }
}
```

上面这段代码，10 个线程各加 1000 次，结果大概率不到 10000。

解决办法有几个方向：1) 尽量不用成员变量，把数据通过方法参数传递。2) 用 `ThreadLocal` 隔离数据，比如保存用户上下文。3) 真要共享状态，就得加锁，或者用 `AtomicInteger` 这种原子类。

像 Spring WebFlux 里的 `WebClient`、`RestTemplate` 这些工具类，官方都明确说可以单例共享，因为它们设计时就避开了实例状态。

所以结论是：**有状态才可能有并发问题**，不是单例导致的，而是共享可变状态 + 缺乏同步造成的。设计时让 Bean 保持无状态，是最简单高效的解法。

## 说下对 Spring MVC 的理解?

Spring MVC 的本质是围绕一个前端控制器 `DispatcherServlet` 展开的，所有请求都先打到它这里，再由它分发给具体的处理方法。整个流程其实就一条链路：请求进来 -> 拦截 -> 映射方法 -> 参数绑定 -> 执行 -> 返回视图或数据。

1) `DispatcherServlet` 接收到请求后，会交给 `HandlerMapping` 去找对应的 Controller 方法。这个映射关系可以是 `@RequestMapping` 注解配置的路径。

2) 找到方法后，`HandlerAdapter` 负责调用它。这时候 Spring 会做参数解析，比如把请求体转成对象、从 URL 提取参数，甚至支持自定义的 `@RequestBody` 或 `@RequestParam`。

3) 方法执行完返回 ModelAndView 或直接写回 JSON (比如用 `@ResponseBody` ), 接着 ViewResolver 可能会渲染页面, 但大多数现代服务都直接输出数据, 前后端分离场景下这步基本跳过。

整个过程中, 拦截器 (Interceptor) 可以在前后插入逻辑, 类似 AOP, 常用来做日志、权限校验。异常统一处理靠 `@ControllerAdvice` 和 `@ExceptionHandler` 捕获全局错误。

代码上最典型的入口就是一个注解类:

```
@RestController
public class OrderController {
    @GetMapping("/order/{id}")
    public Order getOrder(@PathVariable Long id) {
        return orderService.findById(id);
    }
}
```

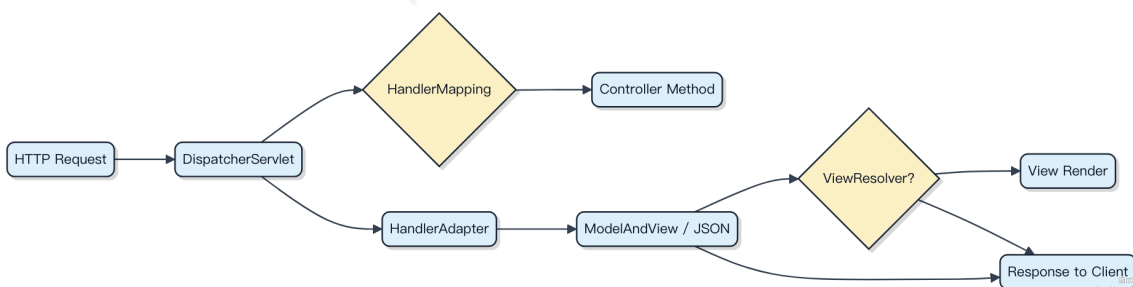
这套模型在 Spring Boot 里被进一步简化, 自动装配让 DispatcherServlet 和组件默认就位, 开发者只关注业务逻辑就行。虽然 WebFlux 出来后有了响应式选择, 但传统 MVC 在项目中还是扛住了大部分 Web 场景, 像电商系统的订单、用户服务这些, 几乎都这么写。

### Spring MVC 具体的工作原理?

Spring MVC 的工作核心是围绕 DispatcherServlet 展开的, 它本质上是一个前端控制器 (Front Controller), 所有请求都先打到它这里, 再由它分发给具体的处理器。

请求进来后, 首先是 DispatcherServlet 接手, 它会委托一系列组件完成映射、适配、处理和渲染。整个流程可以拆成几步来看:

- 1) HandlerMapping 负责根据 URL 找到对应的 Controller 方法。比如你访问 `/user/1`, 框架得知道该调 `UserController` 里的 `getById` 方法。常见的实现有 `RequestMappingHandlerMapping`, 它基于注解扫描路由。
- 2) 找到方法后, HandlerAdapter 出场, 它负责真正去调用那个被 `@RequestMapping` 标记的方法。不同的处理器类型需要不同的适配器, 比如处理简单字符串返回的和处理 `@ResponseBody` 的就不同。
- 3) 方法执行完得到 ModelAndView 或响应数据, 接着 ViewResolver 对视图名进行解析。比如返回 `"userView"`, 它可能被解析成一个 Thymeleaf 模板或 JSP 页面。不过现在前后端分离多, 很多接口直接写 `@ResponseBody`, 这一步就跳过了, 数据直接走 Jackson 序列化 JSON。
- 4) 最后是渲染和响应输出。如果是 REST API, 压根不经过视图渲染, 数据通过 `HttpMessageConverter` 写回客户端。





整个过程里，Interceptor 的拦截逻辑穿插在前后，适合做日志、权限校验这些脏活累活。像 Spring Boot 自动配置把这些组件都默认搭好了，我们写个 @RestController 就能跑起来，背后其实是这套机制在扛住。

## SpringMVC 父子容器是什么知道吗？

SpringMVC 的父子容器其实是个挺关键的设计，很多人用着但没太注意背后的结构。

简单说，Spring 会先启动一个**根容器**，也就是父容器，通常用来放 Service、Repository 这些业务层组件。然后在这个基础上，再为 Web 层启动一个子容器，专门管理 Controller、Interceptor 等 MVC 相关的 Bean。子容器可以访问父容器里的 Bean，反过来不行。

这就意味着你在 Controller 里注入 Service 没问题，因为子容器能看到父容器的东西。但如果反过来，在 Service 里想注入 Controller，启动就直接报错，压根不经过。

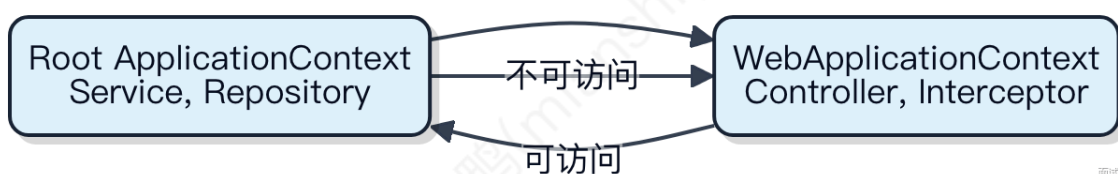
这种设计的好处是职责分离。业务逻辑和 Web 交互解耦，测试的时候也能单独跑业务层，不需要加载整个 Web 环境。像你用 Spring Boot，默认把这事儿给封装掉了，看起来只有一个上下文，但底层还是这套路。

配置上一般这么分：

```
// 父容器: Spring 主配置
@EnableTransactionManagement
@ComponentScan("com.example.service")
class RootConfig { }

// 子容器: MVC 配置
@EnableWebMvc
@ComponentScan("com.example.controller")
class WebConfig implements WebMvcConfigurer { }
```

- 1) 父容器加载非 Web 组件
- 2) 子容器加载 Web 组件
- 3) 子容器持有对父容器的引用，实现层级访问



## 你了解的 Spring 都用到哪些设计模式？

Spring 框架里设计模式用得非常自然，不是为了套模式而套，而是为了解决实际问题。

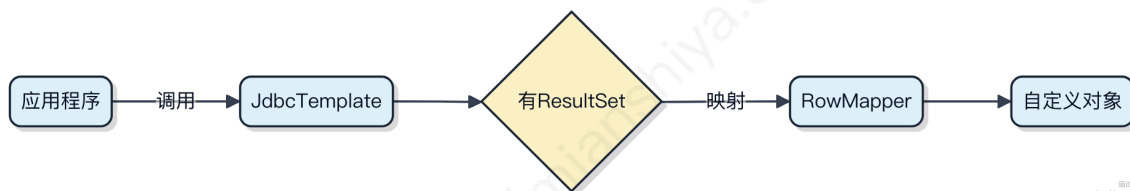
- 1) **单例模式**是应用最广的，默认的 Bean 作用域就是单例。容器启动时创建一次实例，后续获取都返回同一个对象，减少频繁创建销毁的开销。但要注意线程安全，无状态 Bean 没问题，有状态的就得自己管理。
- 2) **工厂模式**体现在 BeanFactory 和 ApplicationContext 上。它们负责实例化、配置、组装 Bean，用户不需要手动 new。比如 XML 或注解定义的 Bean，由工厂统一管理生命周期，实现解耦。
- 3) **代理模式**是 AOP 的基础。Spring 默认用 JDK 动态代理（接口-based），有接口走 Proxy + InvocationHandler；没有接口就切到 CGLIB，生成子类增强方法。事务 @Transactional 就是典型场景，方法前后织入开启/提交事务逻辑。



辑。

4) 模板方法模式在 `JdbcTemplate` 里体现明显。它把获取连接、执行 SQL、处理结果集这些流程定好，把变化的部分——比如结果映射——留给开发者通过 `RowMapper` 实现。

5) 观察者模式对应 Spring 的事件机制。发布 `ApplicationEvent`，监听器 `@EventListener` 接收处理，像上下文刷新、异常通知都可以基于这个链路做扩展。



这些模式不是孤立的，经常组合使用，比如 AOP + 事务 + 代理，形成一套完整的声明式事务解决方案。

### Spring 事务有几个隔离级别？

Spring 事务的隔离级别本质上是对数据库隔离级别的封装，共五种，对应 `TransactionDefinition` 接口中的常量。

#### 1) **DEFAULT**

采用数据库默认的隔离级别，MySQL 是可重复读（`REPEATABLE_READ`），Oracle 是读已提交（`READ_COMMITTED`）。大多数场景下用这个就够了，不用显式指定。

#### 2) **READ\_UNCOMMITTED**

最低隔离级别，允许读未提交数据，可能引发脏读、不可重复读、幻读。基本不会在业务中使用，除非对一致性要求极低。

#### 3) **READ\_COMMITTED**

保证一个事务只能读到其他事务已提交的数据。能避免脏读，但无法避免不可重复读和幻读。PostgreSQL 和 Oracle 默认用这个。

#### 4) **REPEATABLE\_READ**

确保在同一事务中多次读取同一数据结果一致。MySQL InnoDB 默认级别，通过 MVCC 实现，能避免脏读和不可重复读，但理论上仍可能有幻读问题（InnoDB 通过间隙锁缓解）。

#### 5) **SERIALIZABLE**

最高隔离级别，所有事务串行执行。完全避免并发问题，但性能最差，一般只用于强一致性要求的场景，比如金融核心系统。

实际开发中，90% 的情况用 **DEFAULT** 就够了。过度追求高隔离级别反而会带来锁争用、死锁、吞吐下降等问题。Spring 中通过 `@Transactional(isolation = Isolation.REPEATABLE_READ)` 来指定。

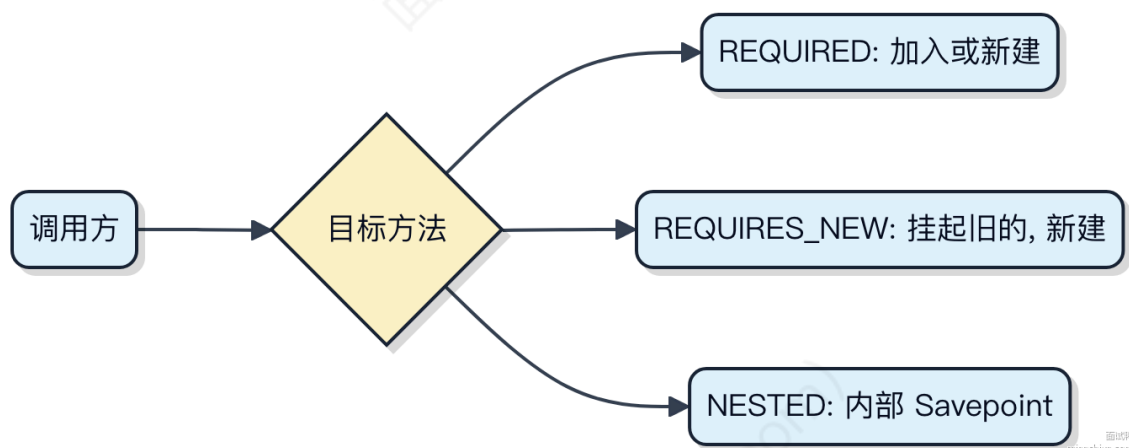


图示从左到右，隔离级别递增，并发性能递减。选哪个级别得看业务对数据一致性和系统吞吐的权衡。

### Spring 有哪几种事务传播行为？

Spring 的事务传播行为决定了多个事务方法相互调用时，事务如何“传播”下去。一共七种，最常用的其实是 **PROPAGATION\_REQUIRED** 和 **PROPAGATION\_REQUIRES\_NEW**。

- 1) **REQUIRED** 当前没事务就新建，有就用已有的。这是默认行为，大部分增删改都走这个。
- 2) **REQUIRES\_NEW** 搞个全新的事务，挂起当前的。适合独立提交的场景，比如记日志、发通知，不能因为主业务回滚就跟着回滚。
- 3) **SUPPORTS** 有事务就加入，没有也正常运行。适合查询类操作，不强制事务。
- 4) **NOT\_SUPPORTED** 不支持事务，总以非事务方式执行。比如调用第三方接口，不想让它卷进当前事务。
- 5) **MANDATORY** 必须在已有事务里运行，没有就抛异常。用于强制约束，比如某个方法只允许在事务中被调用。
- 6) **NEVER** 绝对不要事务，有事务就抛异常。安全控制场景会用到。
- 7) **NESTED** 嵌套事务，外层事务回滚，它也回滚；但它自己回滚不影响外层。基于 Savepoint 实现，注意不是所有数据库都支持。



别死记数字和名字，关键理解每个行为在什么业务场景下适用。比如订单创建中扣库存和发券，发券失败要独立回滚就得用 **REQUIRES\_NEW**。

### Spring 事务传播行为有什么用？

事务传播行为解决的是：当一个已存在的事务方法，被另一个标记了 **@Transactional** 的方法调用时，两者之间该怎么协作。

比如你写了个支付逻辑 **pay()**，它已经加了事务。这时候退款 **refund()** 也用了事务，并且内部调用了 **pay()**，那这两个操作到底是在一个事务里运行，还是各自独立？这就靠传播行为来定。

Spring 定义了 7 种传播行为，最常用的就三种：

#### 1) **REQUIRED**

当前没事务就新建，有就直接加入。这是默认行为。像 **service** 层的主业务方法基本都用这个。

#### 2) **REQUIRES\_NEW**

不管有没有，一律挂起当前事务，新开一个独立事务。适用于日志记录、审计这类需要独立提交的场景，比如用 **RabbitMQ** 发个消息确认，失败也不该影响主流程。

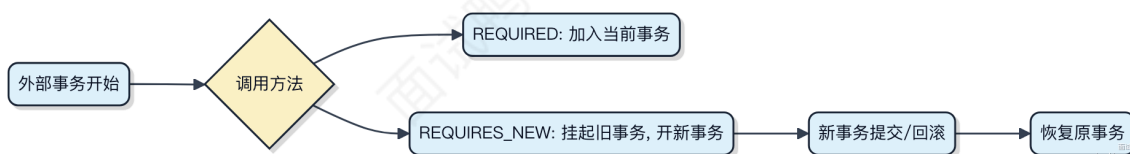
### 3) SUPPORTS

有事务就加入，没有也能运行。适合查询类方法，不强制依赖事务。

代码上看就是注解里的 `propagation` 属性：

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
public void logRecord() {
    // 即使外部有事务，这里也会另起炉灶
}
```

搞不定传播行为，经常会导致数据不一致。比如该隔离的操作被回滚波及，或者该一起提交的反而拆开了。实际开发中，像订单创建过程中扣库存和发券，就得保证在同一个事务里，这时候 `REQUIRED` 就很关键。而做分布式任务调度时，状态更新要独立提交，就得上 `REQUIRES_NEW`。



## Spring 的优点

### 核心是解耦和扩展

Spring 搞的不是从 0 到 1 写功能，而是让已有代码之间别互相咬死。比如业务代码不用自己 new 数据库连接，而是由 Spring 把依赖“塞”进去，这就是 **控制反转** (IoC)。你只管写逻辑，创建和组装对象的事交给容器。

依赖注入让单元测试也轻松了。Mock 一个服务直接塞进去就行，不用启动整个应用。

### AOP 做横切关注点

日志、事务、权限这些逻辑，散在各处会很乱。Spring 的 AOP 能把这些脏活累活抽出来，在方法执行前后动态织入。像用 `@Transactional` 注解的方法，自动套上事务 `begin/commit/rollback`，底层靠的是动态代理。

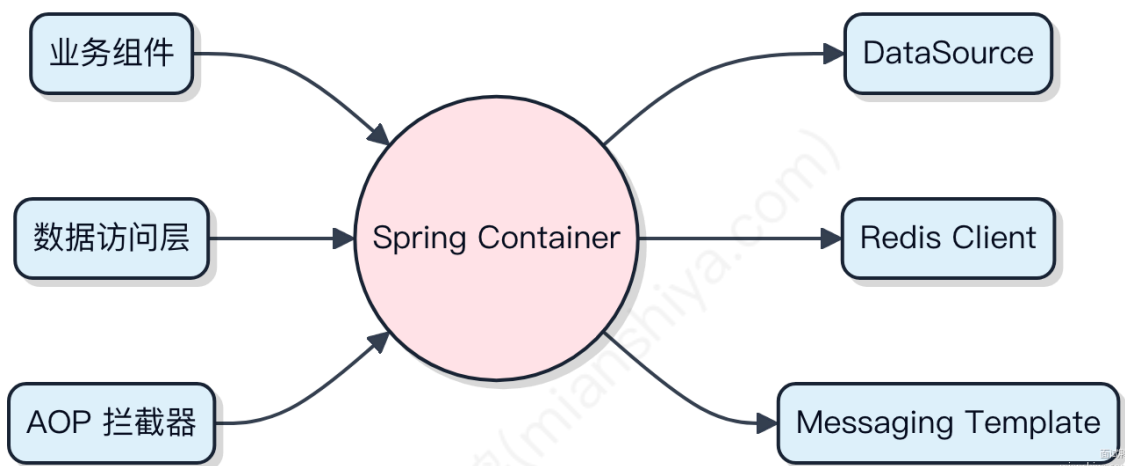
### 生态全，集成简单

你想接 Redis？加个 `spring-boot-starter-data-redis` 就行。要监控？`spring-boot-actuator` 开箱即用。它把主流中间件都包了层，配置几行 yml 就跑起来。像 Kafka、RabbitMQ、Elasticsearch，基本没有对接不了的。

### 容器环境友好

Spring Boot 打出来的 jar 包自带 Tomcat，`java -jar` 直接运行，适合 Docker 部署。启动参数、环境隔离通过 `application.yml` 分 profile 管理，K8s 里配个 configmap 就能切配置。

- 1) IoC 容器管理对象生命周期
- 2) AOP 解决横向逻辑复用
- 3) 自动装配大幅降低配置成本
- 4) 和云原生技术栈无缝衔接



### Spring AOP 相关术语都有哪些？

搞清楚 Spring AOP，先得把几个关键角色拎明白。这些术语不是背概念，而是描述了切面逻辑在运行时是怎么织入的。

**切面 (Aspect)** 是横切关注点的模块化，比如日志、事务。它不是一个接口或类的修饰，而是独立出来的功能块，用 `@Aspect` 标注。

**连接点 (Join Point)** 指程序执行过程中的某个点，比如方法调用或异常抛出。Spring AOP 里只支持方法级别的连接点。

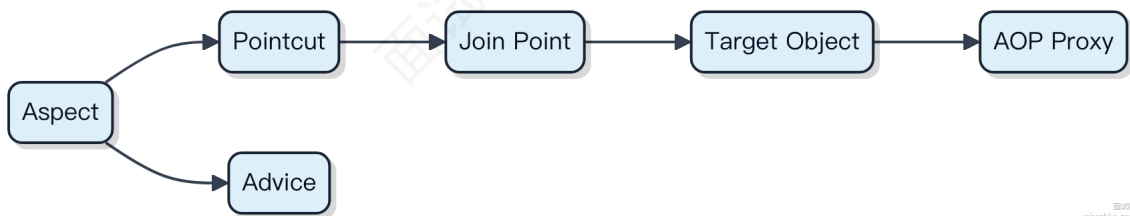
**通知 (Advice)** 就是切面在特定连接点上要执行的动作。按时机分几种：1) 前置通知 (Before) 2) 后置通知 (After) 3) 返回通知 (After-returning) 4) 异常通知 (After-throwing) 5) 环绕通知 (Around) ——最灵活，能控制是否继续执行

**切入点 (Pointcut)** 用来匹配哪些连接点需要被拦截。通常用表达式写，比如 `execution(* com.example.service.*.*(..))` 表示拦截 service 包下所有方法。

**目标对象 (Target Object)** 是被代理的原始对象，也就是你要增强的那个 Bean。

AOP 代理由 Spring 创建，用来实现切面逻辑。默认 JDK 动态代理用于接口，CGLIB 用于类。

**引入 (Introduction)** 允许给目标类添加新方法或字段，这个用得少但确实支持。



整个流程就是：通过 Pointcut 找到哪些 Join Point 要拦截，然后 Advice 定义在那执行什么逻辑，最终代理对象完成织入。

### Spring 通知有哪些类型？

Spring 的通知类型本质上是 AOP 中增强逻辑的执行时机划分，总共就五种，每种对应不同的织入场景。

- 1) 前置通知 (Before Advice) 在目标方法调用前执行，适合做权限校验或日志记录。比如用 `@Before` 注解标记的方法会在切入点方法执行前运行。
- 2) 后置通知 (After Advice) 不管方法是否成功，都会在方法返回或抛出异常后执行，类似 `finally` 块的行为，适合做资源清理。
- 3) 返回通知 (After Returning Advice) 只有在方法正常返回后才触发，可以通过 `returning` 属性获取返回值，常用于结果缓存，比如把查询结果写入 Redis。
- 4) 异常通知 (After Throwing Advice) 仅在方法抛出异常时执行，通过 `throwing` 属性捕获异常对象，适合统一处理业务异常或触发告警，像用 `@AfterThrowing` 捕获 `CustomException`。
- 5) 环绕通知 (Around Advice) 最灵活，能控制整个方法的执行流程，可以决定是否继续执行目标方法，还能修改参数和返回值。性能监控常用它，比如统计接口耗时：

```
@Around("execution(* com.service.*(..))")
public Object measureTime(ProceedingJoinPoint pjp) throws Throwable {
    long start = System.nanoTime();
    try {
        return pjp.proceed();
    } finally {
        log.info("耗时: {} ns", System.nanoTime() - start);
    }
}
```

环绕通知功能强，但别滥用，搞不定的时候优先选前面四种标准通知。

## Spring IOC 容器初始化过程？

Spring IOC 容器启动不是一蹴而就的，它走的是一步一步的生命周期流程。我们以最常见的 `AnnotationConfigApplicationContext` 为例，整个初始化过程可以拆解为几个关键阶段。

- 1) 首先容器会读取配置类，比如你用 `@Configuration` 标记的类，通过反射扫描上面的注解，识别出哪些是需要注册的 Bean。这一步会把配置类本身也作为 `BeanDefinition` 注册进去。
- 2) 然后触发 **BeanFactoryPostProcessor** 的执行，最典型的是 `ConfigurationClassPostProcessor`，它会处理 `@ComponentScan`、`@Import` 等注解，完成 `BeanDefinition` 的补全和注册。这时候你的 `Service`、`Controller` 这些 Bean 就都进来了。
- 3) 接着注册 **BeanPostProcessor**，这些后置处理器会在后续每个 Bean 创建时被回调，比如常用的 `AutowiredAnnotationBeanPostProcessor` 就是在这注册的，为后面的依赖注入做准备。
- 4) 真正开始创建单例 Bean 是在 `finishBeanFactoryInitialization` 方法里，它会遍历所有非懒加载的单例 Bean，调用 `getBean` 触发实例化。这里会走完整的 Bean 生命周期：实例化、属性填充（依赖注入）、初始化方法（`@PostConstruct`、`InitializingBean`）、AOP 代理等。
- 5) 最后发布 `ContextRefreshedEvent`，表示容器刷新完成，应用可以对外提供服务了。



这个过程里，**BeanDefinition** 是核心数据结构，它描述了一个 Bean 的所有元信息，是整个容器初始化的“蓝图”。

## Spring Bean 注册到容器有哪些方式？

Spring 把 Bean 放进容器，其实不光是写个 `@Component` 就完事了，方式挺多的，得看场景。

1) 最常见的是注解驱动，比如用 `@Component`、`@Service` 这些，配合 `@ComponentScan` 扫描自动注册。这是现在最主流的方式，启动时扫描指定包，找到注解标记的类就注册成 Bean 定义。

2) XML 配置虽然老派，但有些老项目还在用。在 `applicationContext.xml` 里写 `<bean class="com.example.User"/>`，容器启动时解析 XML 就把 Bean 定义加载进去了。

3) 编程式注册也很关键，比如你在配置类里写：

```
@Configuration
public class MyConfig {
    @Bean
    public User user() {
        return new User();
    }
}
```

这个方法返回的对象会被注册为单例 Bean。

4) 还有一种是直接操作 API，比如通过 `ApplicationContext` 的子接口 `BeanDefinitionRegistry` 手动注册 Bean 定义。像一些中间件，比如 Feign，就是在启动时动态生成代理类，然后手动注册进去。

5) 最后别忘了条件注册，用 `@Conditional` 注解控制是否注册，Spring Boot 的自动装配全靠它撑着。

这些方式最终都指向一个目标：让 `BeanDefinition` 被加载到 `BeanFactory` 里。容器启动时读取这些定义，后面才好实例化、依赖注入。

要不要画个流程？从配置类/组件扫描到 `BeanDefinitionRegistry` 的过程可以可视化一下。

## Spring 自动装配的方式有哪些？

Spring 的自动装配主要靠注解驱动，不搞 XML 那一套了。现在主流就是基于类路径和条件触发的自动配置。

1) `@Autowired` 是最常用的，按类型 (byType) 注入，字段、方法、构造器都能用。如果同一类型有多个 Bean，得配合 `@Qualifier` 指定名字，不然直接报错。

2) `@Resource` 是 JSR-250 的注解，本质是 byName 优先，找不到再 byType。它属于 Java 标准，不是 Spring 独有，但实际项目里很多人混着用。

3) `@Inject` 来自 JSR-330，功能和 `@Autowired` 基本一样，只是需要额外引入依赖。一般没特殊要求就不用它了。

4) 构造器注入越来越推荐，尤其在使用 Lombok 的 `@RequiredArgsConstructor` 时，代码更干净，还能保证不可变性。

真正让自动装配“动”起来的是 `@EnableAutoConfiguration`，它会去读取 `META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports` 文件里的全限定名列表，把那些带有 `@ConditionalOnXxx` 条件注解的配置类加载进来。比如只有 classpath 有 `RedisTemplate` 才创建 `RedisConfig`。



注意：自动装配不是万能的。像多数据源这种场景，必须手动配置，否则 Spring 压根不知道你想要哪个。条件装配才是自动装配背后的控制逻辑。

## 看过源码吗？说下 Spring 由哪些重要的模块组成？

Spring 框架不是单一的一个库，它是一套组合拳，模块之间各司其职。面试问这个，其实是想看你有没有从整体上理解过它的设计结构。

先说最核心的几个部分。**IoC 容器**是整个 Spring 的基石，对应 `spring-core`、`spring-beans` 和 `spring-context` 这三个模块。没有它们，就没有依赖注入这回事。`spring-core` 提供底层工具类，比如类型转换、资源抽象；`spring-beans` 实现了 Bean 的定义、配置和生命周期管理；而 `spring-context` 在前者基础上扩展出 `ApplicationContext`，支持国际化、事件传播等高级特性。

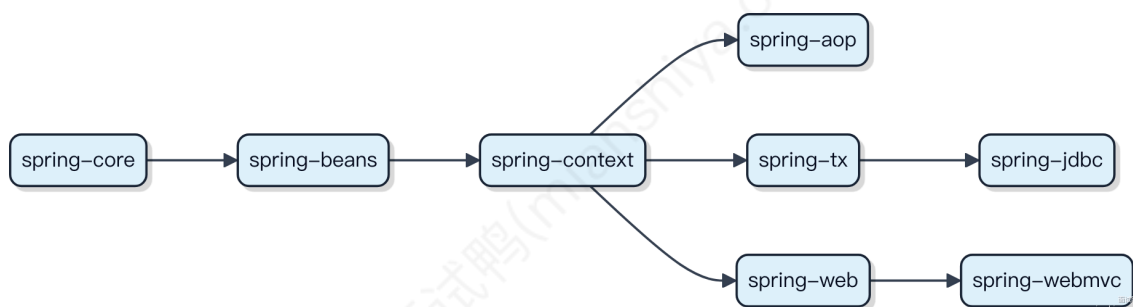
AOP 能力由 `spring-aop` 模块提供，它基于动态代理实现方法级别的拦截。如果你用了 `@Transactional` 或自定义切面，背后就是它在工作。这个模块依赖 `spring-core` 和 `spring-beans`，但可以独立使用。

数据访问方面，`spring-jdbc` 封装了 JDBC 操作，避免你写重复的 try-catch 和资源释放代码。`spring-tx` 统一管理事务，不管是编程式还是声明式事务都靠它支撑。这两个模块让 DAO 层变得干净很多。

Web 开发主要靠 `spring-web` 和 `spring-webmvc`。后者就是我们常说的 Spring MVC，处理 HTTP 请求分发、控制器映射、视图解析这一套流程。现在用 Boot 多了，但底层还是这些模块在驱动。

还有个 `spring-expression`，也就是 SpEL 表达式语言，在注解里写 `#{systemProperties['os.name']}` 这种语法就靠它解析。

这些模块之间有明确的依赖关系，但也能按需引入。比如做批处理可以用 `spring-batch`，做消息可以用 `spring-jms`，都不是必须的。



## 说说 Spring 启动过程？

Spring 启动过程本质是 **IoC 容器的初始化和刷新**，核心入口在 `refresh()` 方法，整个流程大概有 12 个关键步骤。

准备上下文环境时，先设置好环境变量、系统属性，校验必要的配置是否齐全。接着创建 `BeanFactory`，这一步会读取所有 `<bean>` 定义或 `@Component` 注解类，把元信息注册进去，但此时 bean 并没有被实例化。

`BeanFactory` 创建完后，开始加载各种扩展点，比如 `BeanPostProcessor` 和 `ApplicationListener`。这些扩展允许你在 bean 实例化前后插入逻辑，比如 AOP 就是在这里织入代理。



真正的单例 bean 实例化发生在 `finishBeanFactoryInitialization` 这一步。容器会遍历所有非懒加载的单例 bean，触发它们的构造、依赖注入、初始化方法（如 `@PostConstruct`）。循环依赖的解决就发生在这个阶段，Spring 用三级缓存提前暴露刚实例化但未初始化完的对象。

最后一步是发布 `ContextRefreshedEvent`，通知所有监听器上下文已就绪。像 Tomcat 嵌入式容器也是这时候启动，对外提供服务。

代码入口基本都集中在 `AbstractApplicationContext` 的 `refresh()` 方法里：

```
public void refresh() {
    prepareRefresh();
    ConfigurableListableBeanFactory beanFactory = obtainFreshBeanFactory();
    prepareBeanFactory(beanFactory);
    postProcessBeanFactory(beanFactory);
    invokeBeanFactoryPostProcessors(beanFactory);
    registerBeanPostProcessors(beanFactory);
    initMessageSource();
    initApplicationEventMulticaster();
    onRefresh(); // 如 Web 容器启动
    registerListeners();
    finishBeanFactoryInitialization(beanFactory);
    finishRefresh();
}
```

整个过程最关键是 **BeanFactory 准备、扩展注册、bean 实例化、事件发布** 四个阶段。如果某个 bean 初始化失败，比如数据库连不上，容器启动就会直接卡住，应用也就搞不定了。

本资源来自面试鸭：<https://www.mianshiya.com>

推荐更多免费学编程资源：

1. 编程导航学习网站：[学编程、做项目、拿 Offer!](#)
2. 企业高频面试题库：[开始刷题，面试遇原题!](#)
3. 精选简历模板大全：[1 分钟搞定简历!](#)
4. AI 资源导航网站：[获取最新 AI 黑科技!](#)
5. 1 对 1 模拟面试：[随时随地提升面试能力](#)